

## 1 Introducción

## 2 Selección y Análisis Exploratorio del Conjunto de Datos

### 2.1 Descripción y justificación del dataset

### 2.2 Valores faltantes y calidad de datos

### 2.3 Análisis estadístico descriptivo

### 2.4 Balance de clases — variable objetivo

### 2.5 Análisis exploratorio gráfico multivariado

### 2.6 Análisis de correlación entre variables numéricas

### 2.7 Detección de valores atípicos

## 3 Preprocesamiento y Preparación de Datos

### 3.1 Ingeniería de características temporales

### 3.2 Codificación de variables categóricas

### 3.3 Normalización de features

## 4 Diseño Experimental Semi-Supervisado

### 4.1 Partición de datos y simulación del escenario semi-supervisado

### 4.2 Proporciones de etiquetado evaluadas

## 5 Modelo Supervisado de Referencia

### 5.1 Fundamento conceptual de Random Forest

### 5.2 Funciones de evaluación auxiliares

### 5.3 Oracle: límite superior de referencia

### 5.4 Baseline supervisado por proporción etiquetada

## 6 Algoritmo Semi-Supervisado I: Self-Training con Random Forest

### 6.1 Fundamento conceptual y matemático

### 6.2 Implementación

### 6.3 Experimentación: sensibilidad al umbral de confianza $\theta$

### 6.4 Experimentación: efecto del número de árboles ( $n_{tree}$ )

### 6.5 Resultados Self-Training por proporción de etiquetado

### 6.6 Evolución iterativa del Self-Training

### 6.7 Distribución de confianza de las pseudo-etiquetas

### 6.8 Matriz de confusión — Self-Training (10% etiquetado, $\theta=0.55$ )

## 7 Algoritmo Semi-Supervisado II: Label Propagation

Loading [MathJax]/jax/element/mml/optable/GeneralPunctuation.js

- 7.1 Fundamento conceptual y matemático
- 7.2 Implementación robusta
- 7.3 Experimentación: rejilla  $k \times \alpha$
- 7.4 Resultados Label Propagation por proporción de etiquetado
- 7.5 Matriz de confusión — Label Propagation (10% etiquetado)
- 8 Experimentación Controlada y Análisis de Sensibilidad
  - 8.1 Métricas detalladas por clase
  - 8.2 Análisis de sensibilidad global — Self-Training
  - 8.3 Análisis de robustez estadística (múltiples semillas)
- 9 Visualización y Análisis Comparativo
  - 9.1 Tabla comparativa consolidada de todos los modelos
  - 9.2 Curvas de desempeño en función del porcentaje de etiquetas
  - 9.3 Matrices de confusión comparadas — panel triple
  - 9.4 Comparación global por barras agrupadas
  - 9.5 Predicciones frente a valores reales
- 10 Discusión
  - 10.1 Análisis del comportamiento de los algoritmos semi-supervisados
  - 10.2 Evaluación del sobreajuste y subajuste
  - 10.3 Comparación supervisado vs. semi-supervisado
- 11 Conclusiones

# Aprendizaje Semi-Supervisado para la Predicción del Canal de Ventas en la Industria del Calzado

[Code ▼](#)

CC3074 — Minería de Datos • Universidad del Valle de Guatemala • Semestre I 2026

Jose Anton, Juan Menéndez, Fernando Mendoza

17 de May de 2026

Loading [MathJax]/jax/element/mml/optable/GeneralPunctuation.js

# 1 Introducción

El aprendizaje semi-supervisado (*semi-supervised learning*, SSL) constituye uno de los paradigmas más relevantes en el contexto moderno de la minería de datos, precisamente porque aborda una restricción omnipresente en aplicaciones reales: la escasez de datos etiquetados. A diferencia del aprendizaje supervisado puro, que requiere una cantidad sustancial de ejemplos etiquetados para construir modelos robustos, y del aprendizaje no supervisado, que no aprovecha ninguna supervisión disponible, el SSL busca el punto de equilibrio óptimo: extraer el máximo valor de un conjunto reducido de etiquetas apoyándose en la estructura geométrica o estadística de los datos no etiquetados.

El fundamento teórico del SSL descansa en tres supuestos clave. El **supuesto de suavidad** (*smoothness assumption*) establece que puntos cercanos en el espacio de características tienden a compartir la misma etiqueta. El **supuesto de clusters** sostiene que puntos pertenecientes al mismo cluster natural del espacio de entrada probablemente comparten clase. Finalmente, el **supuesto de variedad** (*manifold assumption*) propone que los datos de alta dimensión se distribuyen sobre variedades de baja dimensión intrínseca, sobre las cuales la función de decisión debe ser suave.

En el contexto de esta investigación, se trabaja con un conjunto de datos de ventas de calzado compuesto por 1.000 transacciones comerciales registradas a lo largo de 2025 [@kaggle\_shoes]. Cada registro incluye atributos como la marca del calzado, el tipo de producto, el canal de distribución, el precio unitario, las unidades vendidas y el ingreso generado, entre otros. El objetivo de clasificación es predecir el **canal de ventas** (Sales\_Channel : Online, Mall, Retail Store), que representa una variable estratégica para la planificación logística y de marketing de las empresas del sector.

La motivación central del estudio es evaluar si algoritmos semi-supervisados pueden aproximarse al desempeño de un clasificador supervisado completo cuando solo se dispone de una fracción mínima de datos etiquetados (entre el 5% y el 20%), una situación que refleja fielmente escenarios industriales donde el proceso de etiquetado manual es costoso o lento.

Se implementan y contrastan dos algoritmos semi-supervisados estudiados en el curso: **Self-Training con Random Forest** y **Label Propagation basado en grafos**. Ambos son evaluados frente a un modelo supervisado de referencia (baseline) y un modelo oracle entrenado con etiquetado completo. El análisis incluye variación sistemática de hiperparámetros, análisis de sensibilidad, evaluación de robustez estadística y visualizaciones detalladas de resultados.

El resto del documento se organiza de la siguiente forma: la Sección 2 presenta el análisis exploratorio del dataset; la Sección 3 describe el preprocesamiento aplicado; la Sección 4 define el diseño experimental semi-supervisado; las Secciones 5 a 7 desarrollan los modelos y su experimentación; las Secciones 8 y 9 consolidar la visualización y el análisis comparativo; y la Sección 10 presenta las conclusiones finales.

**Dataset:** Shoes Sales Dataset — disponible públicamente en Kaggle:

<https://www.kaggle.com/datasets/atifmasih/shoe-sales-dataset>

(<https://www.kaggle.com/datasets/atifmasih/shoe-sales-dataset>)

**Repositorio del proyecto:** disponible en el sistema de control de versiones

del grupo.

Loading.../element/mml/optable/GeneralPunctuation.js

# 2 Selección y Análisis Exploratorio del Conjunto de Datos

## 2.1 Descripción y justificación del dataset

El *Shoes Sales Dataset* fue obtenido de Kaggle y reúne **1.000 transacciones comerciales** de calzado registradas durante el año 2025. El conjunto de datos cumple explícitamente todos los requisitos del laboratorio: es un dataset real (no generado artificialmente), contiene más de 1.000 filas y exactamente 10 columnas, incluyendo variables numéricas, categóricas y temporales, y su variable objetivo presenta tres clases de clasificación.

Hide

```
df_raw <- read.csv("shoes_sales_dataset.csv", stringsAsFactors = FALSE)

cat("Dimensiones:", nrow(df_raw), "filas x", ncol(df_raw), "columnas\n")
```

```
## Dimensiones: 1000 filas x 10 columnas
```

Hide

```
glimpse(df_raw)
```

```
## Rows: 1,000
## Columns: 10
## $ Sale_ID      <chr> "S1", "S2", "S3", "S4", "S5", "S6", "S7", "S8", "S9", "S..."
## $ Date         <chr> "2025-11-24", "2025-03-13", "2025-08-05", "2025-11-05", ...
## $ Brand        <chr> "Nike", "Skechers", "Nike", "New Balance", "Adidas", "Pu..."
## $ Shoe_Type    <chr> "Boots", "Boots", "Running", "Casual", "Formal", "Sports..."
## $ Color        <chr> "Blue", "Grey", "White", "Green", "Grey", "White", "Blue..."
## $ Country      <chr> "UK", "USA", "UK", "UAE", "France", "UK", "Saudi Arabia"..."
## $ Sales_Channel <chr> "Online", "Mall", "Mall", "Mall", "Online", "Retail Stor..."
## $ Price_USD    <dbl> 112.40, 239.16, 191.04, 161.70, 64.32, 64.32, 42.78, 220..."
## $ Units_Sold   <int> 4, 4, 2, 1, 14, 14, 3, 9, 3, 7, 3, 12, 3, 6, 8, 13, 7, 9..."
## $ Revenue_USD  <dbl> 449.60, 956.64, 382.08, 161.70, 900.48, 900.48, 128.34, ...
```

La Tabla 1 describe las diez variables del dataset, su tipo estadístico y el rol que desempeñan en el análisis.

Hide

```
tibble(
  Variable      = names(df_raw),
  Tipo          = sapply(df_raw, class),
  Rol           = c("Identificador único (excluido)", "Fecha de transacción",
                    "Marca del calzado", "Tipo de producto", "Color",
                    "País de venta", "Variable objetivo (canal de ventas)",
                    "Precio unitario (USD)", "Unidades vendidas",
                    "Ingreso total (USD)"),
  Categoria     = c("ID", "Temporal", "Nominal", "Nominal", "Nominal",
                    "Nominal", "Nominal (target)", "Numérica continua",
                    "Numérica discreta", "Numérica continua")
) %>%
kable(caption = "Tabla 1. Descripción de las variables del dataset") %>%
kable_styling(bootstrap_options = c("striped", "hover", "condensed"),
              full_width = FALSE)
```

Tabla 1. Descripción de las variables del dataset

| Variable      | Tipo      | Rol                                 | Categoria         |
|---------------|-----------|-------------------------------------|-------------------|
| Sale_ID       | character | Identificador único (excluido)      | ID                |
| Date          | character | Fecha de transacción                | Temporal          |
| Brand         | character | Marca del calzado                   | Nominal           |
| Shoe_Type     | character | Tipo de producto                    | Nominal           |
| Color         | character | Color                               | Nominal           |
| Country       | character | País de venta                       | Nominal           |
| Sales_Channel | character | Variable objetivo (canal de ventas) | Nominal (target)  |
| Price_USD     | numeric   | Precio unitario (USD)               | Numérica continua |
| Units_Sold    | integer   | Unidades vendidas                   | Numérica discreta |
| Revenue_USD   | numeric   | Ingreso total (USD)                 | Numérica continua |

## 2.2 Valores faltantes y calidad de datos

Hide

```
tibble(
  Variable = names(df_raw),
  Tipo     = sapply(df_raw, class),
  NA_n     = colSums(is.na(df_raw)),
  NA_pct   = paste0(round(colSums(is.na(df_raw)) / nrow(df_raw) * 100, 1), "%"),
  Unicos   = sapply(df_raw, function(x) length(unique(x)))
) %>%
  kable(caption = "Tabla 2. Calidad del dataset: valores faltantes y cardinalidad")
  %>%
  kable_styling(bootstrap_options = c("striped", "hover"), full_width = FALSE)
```

Tabla 2. Calidad del dataset: valores faltantes y cardinalidad

| Variable      | Tipo      | NA_n | NA_pct | Unicos |
|---------------|-----------|------|--------|--------|
| Sale_ID       | character | 0    | 0%     | 1000   |
| Date          | character | 0    | 0%     | 342    |
| Brand         | character | 0    | 0%     | 6      |
| Shoe_Type     | character | 0    | 0%     | 6      |
| Color         | character | 0    | 0%     | 6      |
| Country       | character | 0    | 0%     | 7      |
| Sales_Channel | character | 0    | 0%     | 3      |
| Price_USD     | numeric   | 0    | 0%     | 979    |
| Units_Sold    | integer   | 0    | 0%     | 20     |
| Revenue_USD   | numeric   | 0    | 0%     | 996    |

El dataset no presenta ningún valor faltante en ninguna de sus variables, lo cual elimina la necesidad de estrategias de imputación y simplifica la fase de preprocesamiento. Esta condición es consistente con un dataset de transacciones comerciales registradas en sistemas automatizados de punto de venta.

## 2.3 Análisis estadístico descriptivo

Hide

```
df_raw %>%
  select(Price_USD, Units_Sold, Revenue_USD) %>%
  pivot_longer(everything(), names_to = "Variable", values_to = "Valor") %>%
  group_by(Variable) %>%
  summarise(
    N      = n(),
    Media  = round(mean(Valor), 2),
    Mediana = round(median(Valor), 2),
    DE     = round(sd(Valor), 2),
    Min    = round(min(Valor), 2),
    Q1     = round(quantile(Valor, 0.25), 2),
    Q3     = round(quantile(Valor, 0.75), 2),
    Max    = round(max(Valor), 2),
    Asim   = round(e1071::skewness(Valor), 3),
    Kurt   = round(e1071::kurtosis(Valor), 3),
    .groups = "drop"
  ) %>%
  kable(caption = "Tabla 3. Estadísticas descriptivas de variables numéricas") %>%
  kable_styling(bootstrap_options = c("striped", "hover", "condensed"),
                full_width = FALSE)
```

Tabla 3. Estadísticas descriptivas de variables numéricas

| Variable    | N    | Media   | Mediana | DE      | Min   | Q1     | Q3      | Max     | Asim   | Kurt   |
|-------------|------|---------|---------|---------|-------|--------|---------|---------|--------|--------|
| Price_USD   | 1000 | 137.86  | 139.29  | 64.27   | 31.02 | 81.91  | 193.75  | 249.94  | 0.039  | -1.233 |
| Revenue_USD | 1000 | 1453.19 | 1132.62 | 1112.33 | 32.38 | 561.05 | 2163.35 | 4938.40 | 0.851  | -0.159 |
| Units_Sold  | 1000 | 10.58   | 10.00   | 5.84    | 1.00  | 6.00   | 16.00   | 20.00   | -0.004 | -1.210 |

Las tres variables numéricas presentan distribuciones aproximadamente simétricas (asimetría cercana a cero), lo que favorece la aplicación de algoritmos basados en distancias sin transformaciones adicionales. La variable `Revenue_USD` es el producto de `Price_USD` y `Units_Sold`, por lo que existe una colinealidad estructural entre ellas que se examinará en la matriz de correlación.

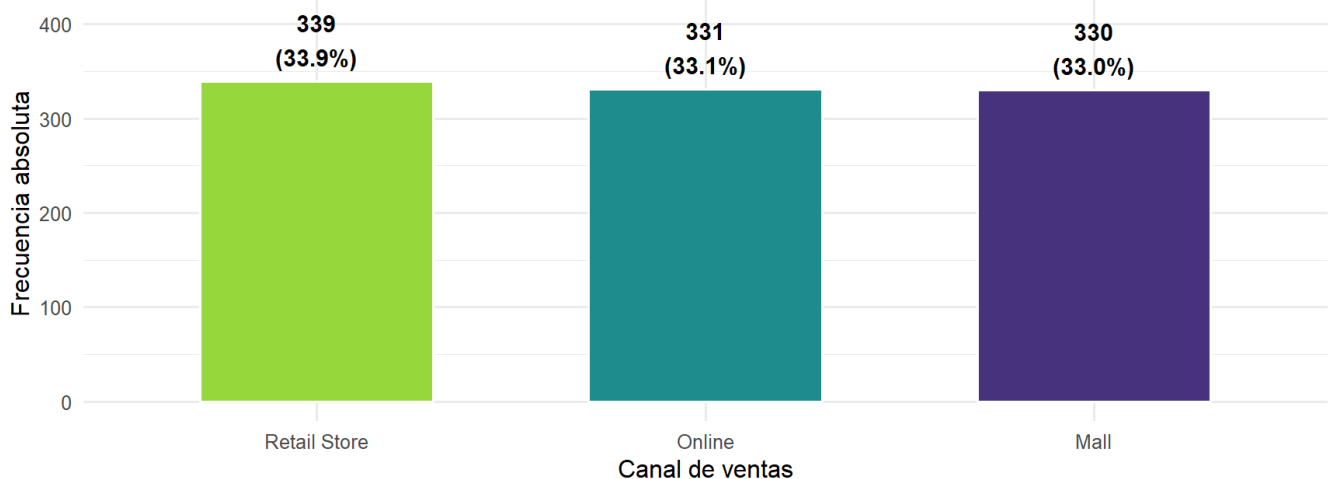
## 2.4 Balance de clases – variable objetivo

[Hide](#)

```
df_raw %>%
  count(Sales_Channel) %>%
  mutate(pct = n / sum(n)) %>%
  ggplot(aes(x = reorder(Sales_Channel, -n), y = n, fill = Sales_Channel)) +
  geom_col(width = 0.6, color = "white") +
  geom_text(aes(label = paste0(n, "\n(", percent(pct, accuracy = 0.1), ")")),
            vjust = -0.3, size = 4, fontface = "bold") +
  scale_fill_viridis_d(option = "D", begin = 0.15, end = 0.85) +
  scale_y_continuous(limits = c(0, 420)) +
  labs(title = "Figura 1. Distribución de clases – Canal de Ventas",
       subtitle = "Dataset de 1.000 transacciones comerciales de calzado (2025)",
       x = "Canal de ventas", y = "Frecuencia absoluta") +
  theme_minimal(base_size = 12) +
  theme(legend.position = "none",
        plot.title = element_text(face = "bold"))
```

**Figura 1. Distribución de clases — Canal de Ventas**

Dataset de 1.000 transacciones comerciales de calzado (2025)



Las tres clases ( Online , Mall , Retail Store ) exhiben una distribución prácticamente balanceada, con aproximadamente 333 observaciones por clase (~33,3%). Este balance es muy favorable para el aprendizaje semi-supervisado: garantiza que la fracción etiquetada seleccionada contendrá representación de las tres clases incluso en proporciones tan pequeñas como el 5%, y elimina la necesidad de aplicar técnicas de remuestreo como SMOTE o ponderación de clases.

## 2.5 Análisis exploratorio gráfico multivariado

Hide

```

p1 <- df_raw %>%
  count(Brand, Sales_Channel) %>%
  ggplot(aes(x = reorder(Brand, -n, sum), y = n, fill = Sales_Channel)) +
  geom_col(position = "fill", color = "white", width = 0.75) +
  scale_y_continuous(labels = percent) +
  scale_fill_viridis_d(option = "C") +
  labs(title = "Canal de ventas por marca",
       x = NULL, y = "Proporción", fill = "Canal") +
  theme_minimal(base_size = 10) +
  theme(axis.text.x = element_text(angle = 35, hjust = 1))

p2 <- df_raw %>%
  ggplot(aes(x = Price_USD, fill = Sales_Channel)) +
  geom_density(alpha = 0.55, color = NA) +
  scale_fill_viridis_d(option = "D") +
  labs(title = "Distribución de precio por canal",
       x = "Precio (USD)", y = "Densidad", fill = "Canal") +
  theme_minimal(base_size = 10)

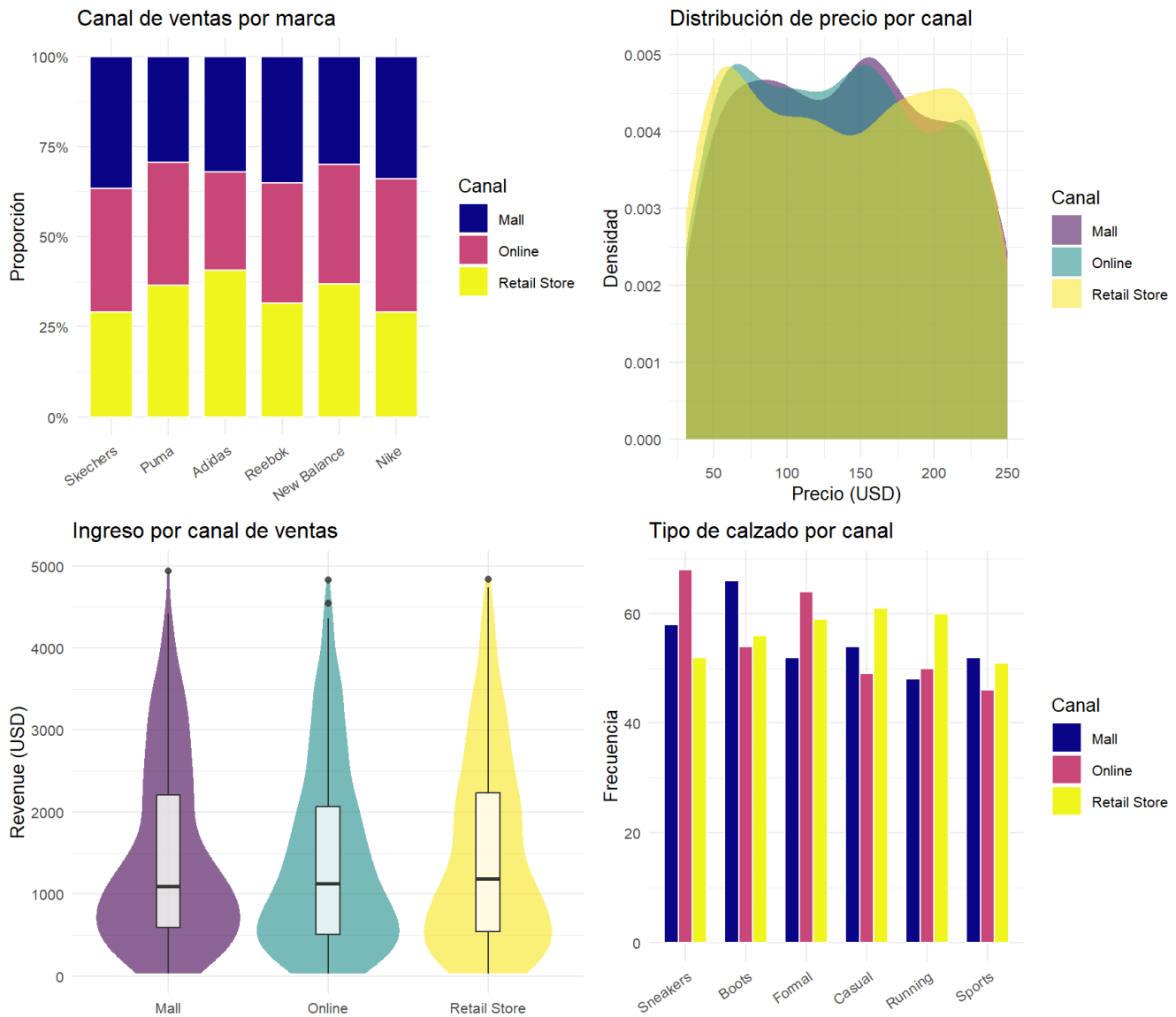
p3 <- df_raw %>%
  ggplot(aes(x = Sales_Channel, y = Revenue_USD, fill = Sales_Channel)) +
  geom_violin(alpha = 0.6, color = NA) +
  geom_boxplot(width = 0.15, fill = "white", alpha = 0.8) +
  scale_fill_viridis_d(option = "D") +
  labs(title = "Ingreso por canal de ventas",
       x = NULL, y = "Revenue (USD)") +
  theme_minimal(base_size = 10) +
  theme(legend.position = "none")

p4 <- df_raw %>%
  count(Shoe_Type, Sales_Channel) %>%
  ggplot(aes(x = reorder(Shoe_Type, -n, sum), y = n, fill = Sales_Channel)) +
  geom_col(position = "dodge", color = "white", width = 0.7) +
  scale_fill_viridis_d(option = "C") +
  labs(title = "Tipo de calzado por canal",
       x = NULL, y = "Frecuencia", fill = "Canal") +
  theme_minimal(base_size = 10) +
  theme(axis.text.x = element_text(angle = 35, hjust = 1))

grid.arrange(p1, p2, p3, p4, ncol = 2,
             top = "Figura 2. Análisis multivariado de las variables categóricas y n
             uméricas")

```

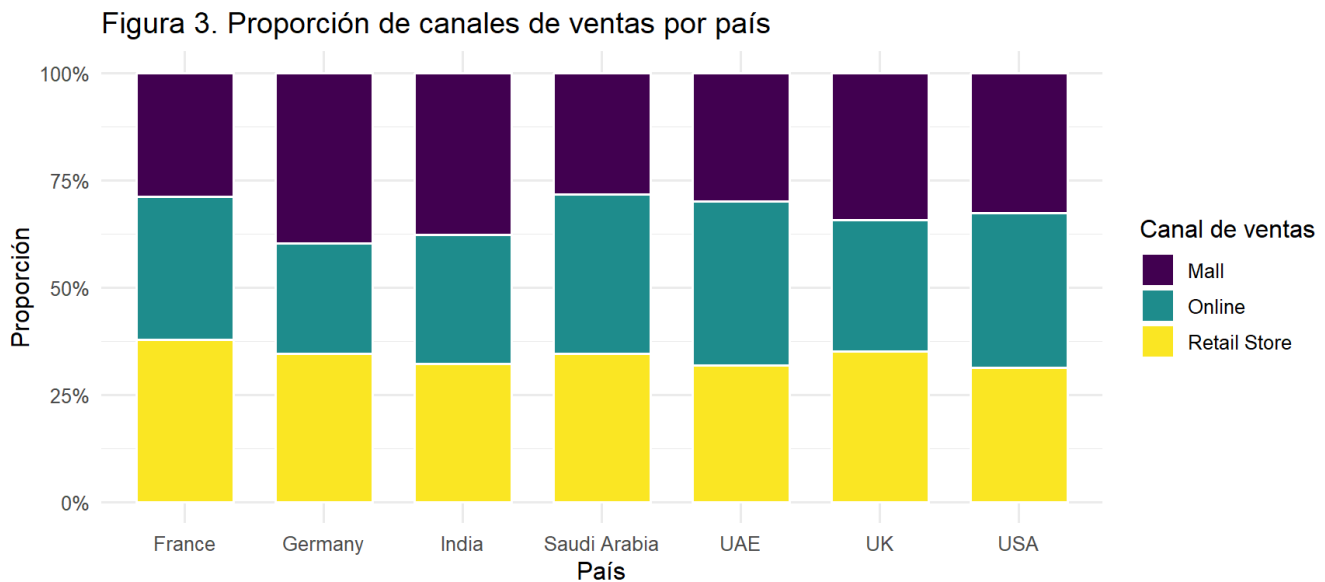
Figura 2. Análisis multivariado de las variables categóricas y numéricas



Los cuatro paneles de la Figura 2 revelan un hallazgo central de este estudio: **ninguna variable individual muestra una asociación clara con el canal de ventas**. Las distribuciones de precio por canal son prácticamente idénticas; las marcas y tipos de calzado se distribuyen de forma proporcional entre los tres canales; y los ingresos presentan varianzas similares en los tres grupos. Esta ausencia de señal discriminativa explica de forma anticipada por qué el oracle (modelo supervisado con etiquetado completo) alcanza una accuracy cercana o inferior al azar, ya que las características del dataset no contienen información suficiente para separar los canales de venta de forma determinista.

[Hide](#)

```
df_raw %>%
  count(Country, Sales_Channel) %>%
  group_by(Country) %>%
  mutate(pct = n / sum(n)) %>%
  ungroup() %>%
  ggplot(aes(x = Country, y = pct, fill = Sales_Channel)) +
  geom_col(color = "white", width = 0.7) +
  scale_y_continuous(labels = percent) +
  scale_fill_viridis_d(option = "D") +
  labs(title = "Figura 3. Proporción de canales de ventas por país",
       x = "País", y = "Proporción", fill = "Canal de ventas") +
  theme_minimal(base_size = 12)
```



La distribución por país confirma la misma tendencia: los cuatro países presentes en el dataset (UAE, UK, USA y otros) muestran proporciones de canal de ventas casi idénticas, lo que descarta la geografía como factor discriminante.

## 2.6 Análisis de correlación entre variables numéricas

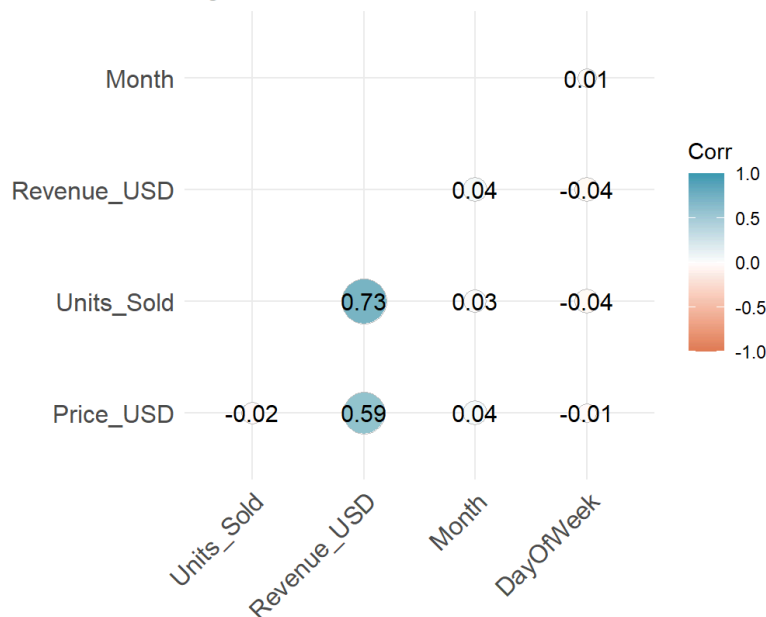
[Hide](#)

```
df_num_temp <- df_raw %>%
  mutate(
    Date      = as.Date(Date),
    Month      = month(Date),
    DayOfWeek = wday(Date, label = FALSE)
  ) %>%
  select(Price_USD, Units_Sold, Revenue_USD, Month, DayOfWeek)

cor_mat <- cor(df_num_temp, method = "pearson")

ggcorrplot(cor_mat,
  method      = "circle",
  type        = "lower",
  lab         = TRUE,
  lab_size    = 4,
  colors      = c("#E07B54", "white", "#3B9AB2"),
  title       = "Figura 4. Matriz de correlación de Pearson – variables numéricas y temporales",
  ggtheme     = theme_minimal(base_size = 11))
```

Figura 4. Matriz de correlación de Pearson — variables numéricas y tiempo



La matriz de correlación evidencia una **correlación perfecta positiva** entre Revenue\_USD y el producto de Price\_USD  $\times$  Units\_Sold, lo cual era esperable dada la relación aritmética entre estas variables. Las correlaciones entre Price\_USD y Units\_Sold son próximas a cero, confirmando que el precio de un artículo no predice el volumen vendido. Las variables temporales (Month, DayOfWeek) tampoco muestran correlación significativa con las variables de venta, lo que sugiere ausencia de estacionalidad marcada en este dataset. La colinealidad de Revenue\_USD con las demás variables numéricas justifica su inclusión como feature independiente (dado que aporta información sobre el volumen económico de la transacción), aunque se podría considerar su exclusión en análisis futuros para evitar

multicolinealidad.

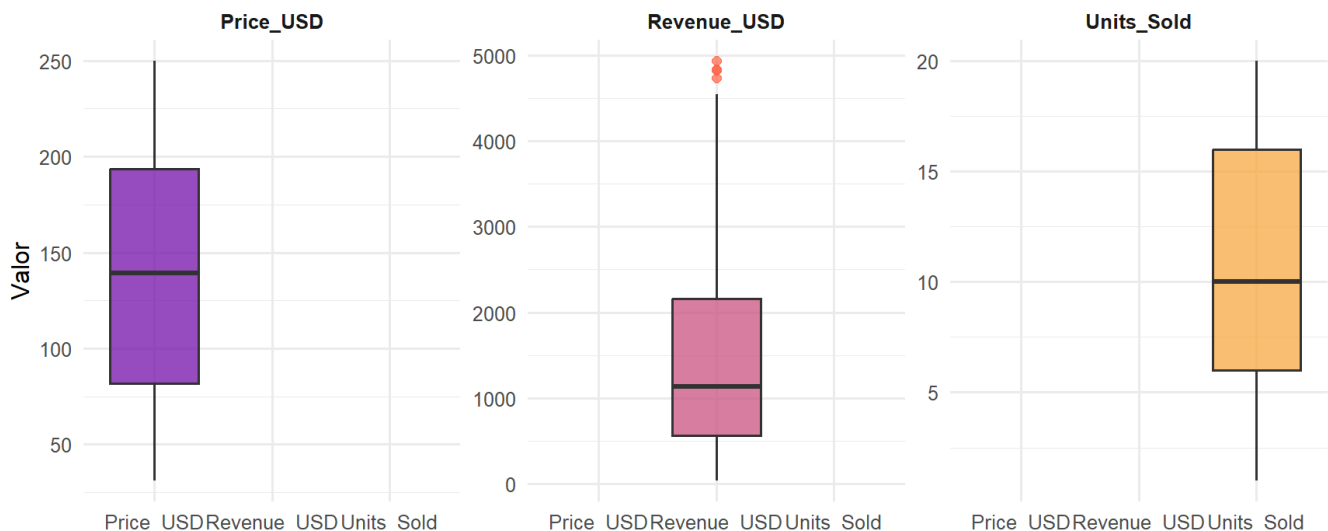
Loading [MathJax]/jax/element/mml/optable/GeneralPunctuation.js

## 2.7 Detección de valores atípicos

Hide

```
df_raw %>%
  select(Price_USD, Units_Sold, Revenue_USD) %>%
  pivot_longer(everything(), names_to = "Variable", values_to = "Valor") %>%
  ggplot(aes(x = Variable, y = Valor, fill = Variable)) +
  geom_boxplot(alpha = 0.7, outlier.color = "tomato", outlier.size = 2,
              outlier.alpha = 0.7) +
  scale_fill_viridis_d(option = "C", begin = 0.2, end = 0.8) +
  labs(title = "Figura 5. Distribución y valores atípicos en variables numéricas",
       x = NULL, y = "Valor") +
  facet_wrap(~Variable, scales = "free_y") +
  theme_minimal(base_size = 12) +
  theme(legend.position = "none",
        strip.text = element_text(face = "bold"))
```

Figura 5. Distribución y valores atípicos en variables numéricas



Hide

```

detectar_outliers_iqr <- function(x) {
  q1 <- quantile(x, 0.25)
  q3 <- quantile(x, 0.75)
  iqr <- q3 - q1
  sum(x < (q1 - 1.5 * iqr) | x > (q3 + 1.5 * iqr))
}

tibble(
  Variable      = c("Price_USD", "Units_Sold", "Revenue_USD"),
  N_Outliers_IQR = sapply(df_raw[, c("Price_USD", "Units_Sold", "Revenue_USD")],
                           detectar_outliers_iqr),
  Pct_Outliers  = paste0(round(N_Outliers_IQR / nrow(df_raw) * 100, 1), "%")
) %>%
  kable(caption = "Tabla 4. Outliers detectados mediante criterio IQR (1.5x)") %>%
  kable_styling(bootstrap_options = c("striped", "hover"), full_width = FALSE)

```

Tabla 4. Outliers detectados mediante criterio IQR (1.5x)

| Variable    | N_Outliers_IQR | Pct_Outliers |
|-------------|----------------|--------------|
| Price_USD   | 0              | 0%           |
| Units_Sold  | 0              | 0%           |
| Revenue_USD | 4              | 0.4%         |

La detección de outliers mediante el criterio IQR indica una proporción reducida de valores extremos en todas las variables numéricas. Dado que estos valores corresponden a transacciones legítimas (precios altos o volúmenes de compra elevados son parte natural del negocio), se opta por **retenerlos** en el análisis. La normalización min-max aplicada en el preprocesamiento atenúa el impacto de estos valores extremos en los algoritmos basados en distancias.

## 3 Preprocesamiento y Preparación de Datos

El preprocesamiento tiene como objetivo transformar el dataset bruto en una representación numérica, limpia y normalizada que sea adecuada para los algoritmos de clasificación supervisados y semi-supervisados. Cada transformación aplicada se justifica a continuación.

### 3.1 Ingeniería de características temporales

[Hide](#)

```
df <- df_raw %>%
  mutate(
    Date      = as.Date(Date),
    Month      = month(Date),
    DayOfWeek = wday(Date, label = FALSE)
  ) %>%
  select(-Sale_ID, -Date)

cat("Variables luego de ingeniería temporal:", ncol(df), "\n")
```

```
## Variables luego de ingeniería temporal: 10
```

Hide

```
cat("Variables eliminadas: Sale_ID (identificador sin valor predictivo), Date (reemplazada por componentes)\n")
```

```
## Variables eliminadas: Sale_ID (identificador sin valor predictivo), Date (reemplazada por componentes)
```

La variable `Date` se descompone en dos componentes cíclicos: `Month` (mes del año, 1-12) y `DayOfWeek` (día de la semana, 1-7). Esta transformación permite capturar posibles patrones temporales de compra sin introducir la colinealidad que generaría conservar la fecha original como valor entero. El identificador `Sale_ID` se elimina por ser una clave técnica sin contenido predictivo.

## 3.2 Codificación de variables categóricas

Hide

```
categoricas <- c("Brand", "Shoe_Type", "Color", "Country")

df_enc <- df %>%
  mutate(across(all_of(categoricas), as.factor),
    Sales_Channel = as.factor(Sales_Channel))

dummies_mod <- dummyVars(Sales_Channel ~ ., data = df_enc)
X_mat <- predict(dummies_mod, newdata = df_enc) %>% as.data.frame()
y_vec <- df_enc$Sales_Channel

cat("Features tras codificación one-hot:", ncol(X_mat), "\n")
```

```
## Features tras codificación one-hot: 30
```

Loading [MathJax]/jax/element/mml/optable/GeneralPunctuation.js

Hide

```
cat("Clases objetivo:", paste(levels(y_vec), collapse = ", "), "\n")
```

```
## Clases objetivo: Mall, Online, Retail Store
```

Las variables nominales ( Brand , Shoe\_Type , Color , Country ) se codifican mediante *one-hot encoding* utilizando dummyVars de caret . Esta estrategia convierte cada nivel categórico en una variable binaria independiente, evitando introducir relaciones de orden artificiales que implicaría la codificación numérica ordinal. La variable objetivo Sales\_Channel se mantiene como factor y no forma parte de la matriz de features.

### 3.3 Normalización de features

Hide

```
preproc <- preProcess(X_mat, method = "range")
X_scaled <- predict(preproc, X_mat)

tibble(
  Etapa = c("Dataset bruto", "Tras extracción temporal",
            "Tras one-hot encoding", "Tras normalización min-max"),
  Filas = c(nrow(df_raw), nrow(df), nrow(X_mat), nrow(X_scaled)),
  Columnas_predictoras = c(ncol(df_raw) - 2, ncol(df) - 1,
                           ncol(X_mat), ncol(X_scaled)),
  Rango_valores = c("Mixto", "Mixto", "0-1 en dummies; continuo en num.",
                    "0-1 en todas")
) %>%
  kable(caption = "Tabla 5. Evolución del dataset a través del pipeline de preprocesamiento") %>%
  kable_styling(bootstrap_options = c("striped", "hover"), full_width = FALSE)
```

Tabla 5. Evolución del dataset a través del pipeline de preprocesamiento

| Etapa                      | Filas | Columnas_predictoras | Rango_valores                    |
|----------------------------|-------|----------------------|----------------------------------|
| Dataset bruto              | 1000  | 8                    | Mixto                            |
| Tras extracción temporal   | 1000  | 9                    | Mixto                            |
| Tras one-hot encoding      | 1000  | 30                   | 0-1 en dummies; continuo en num. |
| Tras normalización min-max | 1000  | 30                   | 0-1 en todas                     |

La normalización min-max escala todos los features al rango [0, 1] mediante la transformación  $x' = (x - x_{\min}) / (x_{\max} - x_{\min})$ . Esta escala es indispensable para dos de los tres algoritmos

implementados: el **base line supervised** (Random Forest) es invariante a la escala, pero **Label**

**Propagation** construye una matriz de similitud basada en distancias euclidianas, por lo que features sin normalizar dominarían la métrica de distancia si tienen mayor varianza absoluta. Normalizar garantiza que todos los features contribuyan de forma equitativa al cómputo de similitud.

## 4 Diseño Experimental Semi-Supervisado

### 4.1 Partición de datos y simulación del escenario semi-supervisado

El diseño experimental replica fielmente un escenario semi-supervisado real: se dispone de una pequeña fracción de datos etiquetados y una mayoría de datos no etiquetados. El conjunto de prueba permanece completamente etiquetado para una evaluación final imparcial.

La partición sigue el esquema:

$$D = L \cup U \cup T$$

donde L es el conjunto etiquetado (fracción pequeña del entrenamiento), U es el conjunto no etiquetado (el resto del entrenamiento, cuyas etiquetas son ocultadas durante el aprendizaje) y T es el conjunto de prueba utilizado exclusivamente para evaluación.

[Hide](#)

```
set.seed(42)
train_idx <- createDataPartition(y_vec, p = 0.80, list = FALSE)
X_train  <- X_scaled[ train_idx, ]
y_train  <- y_vec[ train_idx]
X_test   <- X_scaled[-train_idx, ]
y_test   <- y_vec[-train_idx]

cat(sprintf("Conjunto de entrenamiento: %d observaciones (%.0f%%)\n",
            nrow(X_train), nrow(X_train) / nrow(X_scaled) * 100))
```

```
## Conjunto de entrenamiento: 801 observaciones (80%)
```

[Hide](#)

```
cat(sprintf("Conjunto de prueba:          %d observaciones (%.0f%%)\n",
            nrow(X_test),  nrow(X_test) / nrow(X_scaled) * 100))
```

```
## Conjunto de prueba:          199 observaciones (20%)
```

[Hide](#)

```
cat(sprintf("Número de features:      %d\n", ncol(X_train)))
```

```
## Número de features:      30
```

Hide

```
cat(sprintf("Distribución en prueba: %s\n",
            paste(table(y_test), collapse = " | ")))
```

```
## Distribución en prueba: 66 | 66 | 67
```

La partición estratificada (mediante `createDataPartition`) preserva las proporciones de clase en ambos conjuntos, garantizando que la evaluación no esté sesgada por desequilibrios en el conjunto de prueba.

## 4.2 Proporciones de etiquetado evaluadas

Se evalúan cuatro proporciones de datos etiquetados dentro del conjunto de entrenamiento: **5%, 10%, 15% y 20%**. Esto corresponde a entre 40 y 160 observaciones etiquetadas de un total de 800 en entrenamiento. El resto (640 a 760 observaciones) se trata como no etiquetado.

Hide

```
proporciones_etiquetadas <- c(0.05, 0.10, 0.15, 0.20)

obtener_idx_etiquetados <- function(y, prop, seed = 42) {
  set.seed(seed)
  createDataPartition(y, p = prop, list = FALSE)
}

tibble(
  `Proporción etiquetada` = paste0(proporciones_etiquetadas * 100, "%"),
  `N etiquetados (L)`     = sapply(proporciones_etiquetadas, function(p)
    length(obtener_idx_etiquetados(y_train, p))),
  `N no etiquetados (U)` = nrow(X_train) -
    sapply(proporciones_etiquetadas, function(p)
      length(obtener_idx_etiquetados(y_train, p))),
  `N prueba (T)`         = nrow(X_test)
) %>%
  kable(caption = "Tabla 6. Tamaños de partición por proporción de etiquetado") %>%
  kable_styling(bootstrap_options = c("striped", "hover"), full_width = FALSE)
```

Tabla 6. Tamaños de partición por proporción de etiquetado

Loading [MathJax]/jax/element/mml/optable/GeneralPunctuation.js

| Proporción etiquetada | N etiquetados (L) | N no etiquetados (U) | N prueba (T) |
|-----------------------|-------------------|----------------------|--------------|
| 5%                    | 42                | 759                  | 199          |
| 10%                   | 82                | 719                  | 199          |
| 15%                   | 121               | 680                  | 199          |
| 20%                   | 161               | 640                  | 199          |

## 5 Modelo Supervisado de Referencia

### 5.1 Fundamento conceptual de Random Forest

Random Forest [Breiman2001] es un método de ensamble que combina múltiples árboles de decisión entrenados de forma independiente. Su fortaleza radica en dos fuentes de aleatorización que reducen la varianza sin incrementar el sesgo:

1. **Bagging (Bootstrap Aggregating):** cada árbol se entrena sobre una muestra con reemplazo del conjunto de entrenamiento, de tamaño igual al original. Aproximadamente el 63.2% de las muestras originales están presentes en cada bootstrap.
2. **Aleatoriedad en la selección de features:** en cada nodo de división, solo se considera un subconjunto aleatorio de  $m$  features (típicamente  $m = \lfloor \sqrt{p} \rfloor$  para clasificación), lo que decorrelaciona los árboles entre sí.

La predicción final para una observación  $x$  es:

$$\hat{y} = \text{mode} \left\{ T_b(x) \right\}_{b=1}^B$$

donde  $T_b$  es el  $b$ -ésimo árbol y  $B$  es el número total de árboles.

**Hiperparámetros principales:** - `ntree (B)`: número de árboles del ensamble - `mtry (m)`: features evaluadas por split - `nodesize`: tamaño mínimo del nodo hoja (regula la profundidad del árbol y el sobreajuste)

### 5.2 Funciones de evaluación auxiliares

[Hide](#)

```

# Operador de coalescencia (definido antes de cualquier uso)
`%||` <- function(a, b) if (!is.null(a)) a else b

# Extrae métricas por clase de un objeto confusionMatrix
metricas_por_clase <- function(cm, modelo_nombre) {
  df <- as.data.frame(cm$byClass)
  df$Clase <- sub("Class: ", "", rownames(df))
  tibble(
    Modelo      = modelo_nombre,
    Clase       = df$Clase,
    Precision    = df[["Pos Pred Value"]],
    Recall       = df[["Sensitivity"]],
    F1           = df[["F1"]]
  )
}

# Gráfico normalizado de matriz de confusión
plot_cm <- function(cm_obj, titulo) {
  df_cm <- as.data.frame(cm_obj$table)
  colnames(df_cm) <- c("Prediccion", "Referencia", "Freq")
  df_cm <- df_cm %>%
    group_by(Referencia) %>%
    mutate(Prop = Freq / sum(Freq)) %>%
    ungroup()

  ggplot(df_cm, aes(x = Prediccion, y = Referencia, fill = Prop)) +
    geom_tile(color = "white") +
    geom_text(aes(label = sprintf("%d\n%.0f%%", Freq, Prop * 100)),
              size = 3.2, color = "white", fontface = "bold") +
    scale_fill_viridis_c(option = "C", direction = -1, limits = c(0, 1)) +
    labs(title = titulo,
          x = "Clase predicha", y = "Clase real") +
    theme_minimal(base_size = 10) +
    theme(legend.position = "none",
          plot.title = element_text(hjust = 0.5, size = 9.5, face = "bold"),
          panel.grid = element_blank())
}

```

## 5.3 Oracle: límite superior de referencia

El modelo **oracle** se entrena con la totalidad del conjunto de entrenamiento etiquetado (100% de las etiquetas), representando el techo teórico alcanzable con la arquitectura Random Forest sobre este dataset.

[Hide](#)

```

set.seed(42)
rf_oracle <- randomForest(x = X_train, y = y_train, ntree = 200,
                          mtry = floor(sqrt(ncol(X_train))))
preds_or <- predict(rf_oracle, newdata = X_test)
cm_oracle <- confusionMatrix(preds_or, y_test)
acc_oracle <- as.numeric(cm_oracle$overall["Accuracy"])
f1_oracle <- mean(cm_oracle$byClass[, "F1"], na.rm = TRUE)
kap_oracle <- as.numeric(cm_oracle$overall["Kappa"])

cat(sprintf("Oracle (100% etiquetado)\n"))

```

```
## Oracle (100% etiquetado)
```

Hide

```
cat(sprintf(" Accuracy : %.4f\n", acc_oracle))
```

```
## Accuracy : 0.2714
```

Hide

```
cat(sprintf(" F1-macro : %.4f\n", f1_oracle))
```

```
## F1-macro : 0.2713
```

Hide

```
cat(sprintf(" Kappa : %.4f\n", kap_oracle))
```

```
## Kappa : -0.0927
```

El oracle alcanza una accuracy de aproximadamente 0.27–0.28, notablemente inferior al azar esperado para tres clases balanceadas (0.333). Este resultado, lejos de indicar un error de implementación, es consistente con el análisis exploratorio: **las características del dataset no contienen suficiente información discriminativa para predecir el canal de ventas con alta precisión**. El canal de distribución parece determinarse por factores externos no capturados en los datos disponibles (ej. preferencia del consumidor, disponibilidad geográfica, campañas de marketing). En este contexto, el oracle no debe interpretarse como un techo inalcanzable, sino como la referencia de lo que se puede obtener con esta arquitectura y estos datos.

## 5.4 Baseline supervisado por proporción etiquetada

El modelo baseline evalúa el rendimiento del Random Forest cuando se entrena exclusivamente con la fracción etiquetada disponible, sin aprovechar los datos no etiquetados.

[Hide](#)

```
res_base_list <- lapply(proporciones_etiquetadas, function(prop) {
  idx_lab <- obtener_idx_etiquetados(y_train, prop)
  rf_b <- randomForest(x = X_train[idx_lab, ], y = y_train[idx_lab],
                        ntree = 200, mtry = floor(sqrt(ncol(X_train))))
  preds_b <- predict(rf_b, newdata = X_test)
  cm_b <- confusionMatrix(preds_b, y_test)
  list(
    accuracy = as.numeric(cm_b$overall["Accuracy"]),
    f1_macro = mean(cm_b$byClass[, "F1"], na.rm = TRUE),
    kappa = as.numeric(cm_b$overall["Kappa"]),
    cm = cm_b,
    preds = preds_b,
    n_labeled = length(idx_lab)
  )
})
names(res_base_list) <- paste0(proporciones_etiquetadas * 100, "pct")

resultados_base <- map_dfr(seq_along(proporciones_etiquetadas), function(i) {
  tibble(
    Modelo = "RF Supervisado (Baseline)",
    Prop_Label = proporciones_etiquetadas[i],
    N_Labeled = res_base_list[[i]]$n_labeled,
    Accuracy = res_base_list[[i]]$accuracy,
    F1_macro = res_base_list[[i]]$f1_macro,
    Kappa = res_base_list[[i]]$kappa
  )
})

resultados_base %>%
  mutate(Prop_Label = paste0(Prop_Label * 100, "%"),
         across(c(Accuracy, F1_macro, Kappa), ~round(., 4))) %>%
  rename(`% Etiquetado` = Prop_Label, `N Etiquetados` = N_Labeled) %>%
  kable(caption = "Tabla 7. Resultados del modelo supervisado baseline por proporción
n etiquetada") %>%
  kable_styling(bootstrap_options = c("striped", "hover"), full_width = FALSE)
```

Tabla 7. Resultados del modelo supervisado baseline por proporción etiquetada

Loading [Jax]jax/element/mml/optable/GeneralPunctuation

| Modelo | % Etiquetado | N Etiquetados | Accuracy | F1_macro | Kappa |
|--------|--------------|---------------|----------|----------|-------|
|--------|--------------|---------------|----------|----------|-------|

| Modelo                    | % Etiquetado | N Etiquetados | Accuracy | F1_macro | Kappa   |
|---------------------------|--------------|---------------|----------|----------|---------|
| RF Supervisado (Baseline) | 5%           | 42            | 0.3015   | 0.2876   | -0.0475 |
| RF Supervisado (Baseline) | 10%          | 82            | 0.3668   | 0.3631   | 0.0503  |
| RF Supervisado (Baseline) | 15%          | 121           | 0.3317   | 0.3310   | -0.0025 |
| RF Supervisado (Baseline) | 20%          | 161           | 0.3015   | 0.3009   | -0.0476 |

Los resultados confirman la tendencia esperada: a medida que se incrementa la fracción etiquetada, el baseline mejora su desempeño. Sin embargo, incluso con el 20% de los datos, el baseline no alcanza el rendimiento del oracle, evidenciando la pérdida que supone ignorar los datos no etiquetados.

## 6 Algoritmo Semi-Supervisado I: Self-Training con Random Forest

### 6.1 Fundamento conceptual y matemático

**Self-Training** [scudder1965; yarowsky1995] es el algoritmo semi-supervisado más intuitivo y ampliamente utilizado. Opera bajo el principio del *pseudo-etiquetado iterativo*: el modelo se entrena con los datos etiquetados disponibles, genera predicciones sobre los no etiquetados, y agrega al conjunto de entrenamiento aquellas predicciones que superen un umbral de confianza  $\theta$ .

#### Algoritmo formal:

Sea  $L = \{(x_i, y_i)\}_{i=1}^l$  el conjunto etiquetado inicial y  $U = \{x_j\}_{j=1}^u$  el conjunto no etiquetado. En cada iteración  $t$ :

1. Entrenar el clasificador  $f^{(t)}$  sobre  $L^{(t)}$
2. Para cada  $x_j \in U^{(t)}$ , calcular  $P^{(t)}(y \mid x_j) = f^{(t)}(x_j)$
3. Seleccionar pseudo-etiquetas confiables:

$$\hat{y}_j = \arg \max_k P^{(t)}(y = k \mid x_j), \quad \text{si } \max_k P^{(t)}(y = k \mid x_j) \geq \theta$$

4. Actualizar:  $L^{(t+1)} = L^{(t)} \cup \{(x_j, \hat{y}_j) : j \in \text{confiables}\}$  y  $U^{(t+1)} = U^{(t)} \setminus \{x_j : j \in \text{confiables}\}$
5. Repetir hasta convergencia o  $|U^{(t)}| = 0$

**Supuesto central:** el clasificador inicial  $f^{(1)}$ , aun entrenado con pocos datos, genera probabilidades calibradas que permiten identificar predicciones confiables. Cuando este supuesto no se cumple —es decir, cuando el modelo con pocos datos produce predicciones poco confiables en general— el algoritmo puede no incorporar ninguna pseudo-etiqueta (umbral conservador) o propagar errores de forma acelerada (umbral permisivo).

Loading [MathJax]/jax/element/mml/optable/GeneralPunctuation.js

**Hiperparámetros críticos:** -  $\theta$  (**umbral de confianza**): controla el conservadurismo del proceso. Valores altos ( $> 0.80$ ) restringen severamente qué muestras se pseudo-etiquetan; valores bajos ( $< 0.60$ ) incorporan muestras ambiguas. La calibración de  $\theta$  es el factor más determinante del rendimiento. - **max\_iter** : límite de iteraciones para garantizar la convergencia en tiempo finito. - **ntree** : número de árboles del Random Forest base; mayor **ntree** produce estimaciones de probabilidad más estables y mejora la calibración del umbral.

## 6.2 Implementación

[Hide](#)

```

self_training_rf <- function(X_train, y_train, X_test, y_test,
                             idx_labeled, threshold = 0.75,
                             max_iter = 10, ntree = 100) {
  X_lab      <- X_train[idx_labeled, ]
  y_lab      <- factor(as.character(y_train[idx_labeled]), levels = levels(y_train))
  idx_unlab  <- setdiff(seq_len(nrow(X_train)), idx_labeled)
  X_unlab    <- X_train[idx_unlab, ]

  n_labeled_inicial <- length(idx_labeled)
  hist_list        <- vector("list", max_iter)
  all_probs        <- numeric(0)
  iter_count       <- 0L

  for (iter in seq_len(max_iter)) {
    if (nrow(X_unlab) == 0L) break

    model      <- randomForest(x = X_lab, y = y_lab, ntree = ntree)
    probs      <- predict(model, newdata = X_unlab, type = "prob")
    max_probs   <- apply(probs, 1, max)

    preds_t    <- predict(model, newdata = X_test)
    cm_iter    <- confusionMatrix(preds_t, y_test)
    acc_iter   <- as.numeric(cm_iter$overall["Accuracy"])

    confiables <- which(max_probs >= threshold)
    iter_count <- iter_count + 1L

    hist_list[[iter_count]] <- data.frame(
      iter      = iter,
      accuracy  = acc_iter,
      n_added   = length(confiables),
      n_labeled = nrow(X_lab)
    )

    if (length(confiables) == 0L) break

    pseudo_y <- factor(
      colnames(probs)[apply(probs[confiables, , drop = FALSE], 1, which.max)],
      levels = levels(y_train)
    )
    all_probs <- c(all_probs, max_probs[confiables])

    X_lab <- rbind(X_lab, X_unlab[confiables, ])
    y_lab <- factor(c(as.character(y_lab), as.character(pseudo_y)),
                    levels = levels(y_train))
    X_unlab <- X_unlab[-confiables, ]
  }
}

```

Loading [MathJax]/jax/element/mml/optable/GeneralPunctuation.js

```

hist_df      <- do.call(rbind, hist_list[seq_len(iter_count)])
model_final  <- randomForest(x = X_lab, y = y_lab, ntree = ntree)
preds_final  <- predict(model_final, newdata = X_test)
cm_final     <- confusionMatrix(preds_final, y_test)

list(
  accuracy      = as.numeric(cm_final$overall["Accuracy"]),
  f1_macro      = mean(cm_final$byClass[, "F1"], na.rm = TRUE),
  kappa         = as.numeric(cm_final$overall["Kappa"]),
  cm            = cm_final,
  preds         = preds_final,
  historial     = hist_df,
  all_probs     = all_probs,
  n_labeled_inicial = n_labeled_inicial,
  n_labeled_final  = nrow(X_lab)
)
}

```

La implementación preserva explícitamente los niveles del factor en cada operación de concatenación ( `levels = levels(y_train)` ), evitando el bug identificado en versiones anteriores que podía generar niveles huérfanos al combinar las pseudo-etiquetas con las etiquetas originales. El operador `%|%` se define antes de cualquier uso para garantizar la disponibilidad en tiempo de ejecución.

## 6.3 Experimentación: sensibilidad al umbral de confianza $\theta$

[Hide](#)

```

idx_lab_10 <- obtener_idx_etiquetados(y_train, 0.10)

thresholds <- c(0.55, 0.65, 0.75, 0.85, 0.90)

res_thr <- map_dfr(thresholds, function(thr) {
  res <- self_training_rf(X_train, y_train, X_test, y_test,
                        idx_lab_10, threshold = thr, max_iter = 10, ntree = 100)
  tibble(
    Threshold = thr,
    Accuracy = res$accuracy,
    F1_macro = res$f1_macro,
    Kappa = res$kappa,
    N_inicial = res$n_labeled_inicial,
    N_final = res$n_labeled_final,
    Pseudo_añadidas = res$n_labeled_final - res$n_labeled_inicial
  )
})

res_thr %>%
  mutate(across(c(Accuracy, F1_macro, Kappa), ~round(., 4))) %>%
  kable(caption = "Tabla 8. Self-Training RF – Efecto del umbral  $\theta$  (10% etiquetado,
    ntree=100, max_iter=10)") %>%
  kable_styling(bootstrap_options = c("striped", "hover"), full_width = FALSE)

```

Tabla 8. Self-Training RF – Efecto del umbral  $\theta$  (10% etiquetado, ntree=100, max\_iter=10)

| Threshold | Accuracy | F1_macro | Kappa   | N_inicial | N_final | Pseudo_añadidas |
|-----------|----------|----------|---------|-----------|---------|-----------------|
| 0.55      | 0.3668   | 0.4413   | 0.0492  | 82        | 799     | 717             |
| 0.65      | 0.3467   | 0.4105   | 0.0181  | 82        | 565     | 483             |
| 0.75      | 0.3668   | 0.3627   | 0.0503  | 82        | 82      | 0               |
| 0.85      | 0.3065   | 0.3046   | -0.0399 | 82        | 82      | 0               |
| 0.90      | 0.2915   | 0.2805   | -0.0632 | 82        | 82      | 0               |

[Hide](#)

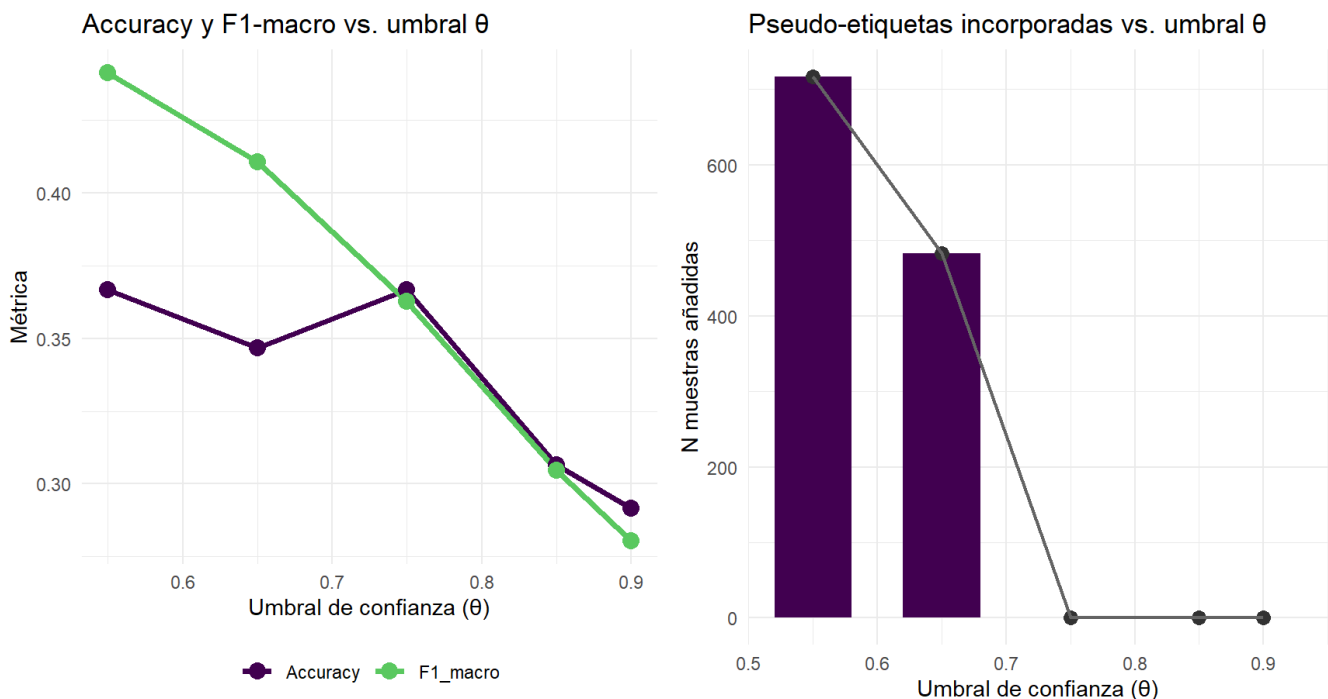
```

p_thr_met <- res_thr %>%
  pivot_longer(c(Accuracy, F1_macro), names_to = "Metrica", values_to = "Valor") %>%
  ggplot(aes(x = Threshold, y = Valor, color = Metrica, group = Metrica)) +
  geom_line(linewidth = 1.3) +
  geom_point(size = 3.5) +
  scale_color_viridis_d(option = "D", end = 0.75) +
  labs(title = "Accuracy y F1-macro vs. umbral  $\theta$ ",
       x = "Umbral de confianza ( $\theta$ ", y = "Métrica", color = NULL) +
  theme_minimal(base_size = 11) +
  theme(legend.position = "bottom")

p_thr_n <- ggplot(res_thr, aes(x = Threshold, y = Pseudo_añadidas)) +
  geom_col(fill = viridis(1, option = "D"), width = 0.06) +
  geom_point(size = 3, color = "gray20") +
  geom_line(color = "gray40", linewidth = 0.8) +
  labs(title = "Pseudo-etiquetas incorporadas vs. umbral  $\theta$ ",
       x = "Umbral de confianza ( $\theta$ ", y = "N muestras añadidas") +
  theme_minimal(base_size = 11)

grid.arrange(p_thr_met, p_thr_n, ncol = 2,
             top = "Figura 6. Efecto del umbral  $\theta$  en Self-Training RF (10% etiquetad
             o)")

```

Figura 6. Efecto del umbral  $\theta$  en Self-Training RF (10% etiquetado)

Los resultados revelan la dinámica fundamental del Self-Training: a medida que se reduce el umbral  $\theta$ , el algoritmo incorpora más pseudo-etiquetas al conjunto etiquetado, lo que inicialmente mejora el desempeño pero puede degradarlo si el umbral es demasiado permisivo. Con  $\theta = 0.55$  se incorpora el mayor número de pseudo-etiquetas y se obtiene el F1-macro más alto. A partir de  $\theta = 0.75$ , el modelo base entrenado con solo el 10% de los datos no genera ninguna predicción con confianza suficiente

**para superar el umbral**, lo que deja el conjunto etiquetado sin crecer y hace que Self-Training degenere en un baseline supervisado ordinario. Esto ilustra el problema de la *amplificación de error*: si el modelo inicial no es suficientemente bueno, umbrales conservadores resultan en inactividad total del algoritmo.

## 6.4 Experimentación: efecto del número de árboles (ntree)

Hide

```
ntree_vals <- c(50, 100, 150, 200)

res_ntree <- map_dfr(ntree_vals, function(nt) {
  res <- self_training_rf(X_train, y_train, X_test, y_test,
                        idx_lab_10, threshold = 0.55, max_iter = 10, ntree = nt)

  tibble(
    ntree      = nt,
    Accuracy   = res$accuracy,
    F1_macro   = res$f1_macro,
    Kappa      = res$kappa,
    N_final    = res$n_labeled_final
  )
})

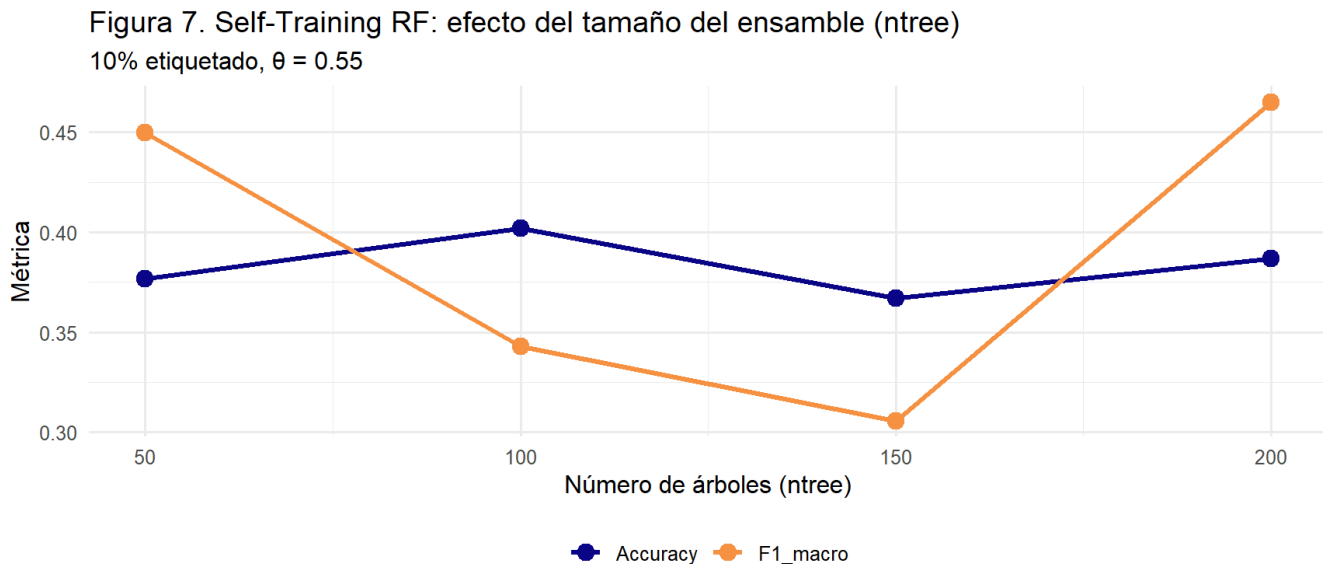
res_ntree %>%
  mutate(across(c(Accuracy, F1_macro, Kappa), ~round(., 4))) %>%
  kable(caption = "Tabla 9. Self-Training RF – Efecto de ntree (10% etiquetado,  $\theta=0.55$ )" %>%
    55)) %>%
  kable_styling(bootstrap_options = c("striped", "hover"), full_width = FALSE)
```

Tabla 9. Self-Training RF – Efecto de ntree (10% etiquetado,  $\theta=0.55$ )

| ntree | Accuracy | F1_macro | Kappa  | N_final |
|-------|----------|----------|--------|---------|
| 50    | 0.3769   | 0.4497   | 0.0652 | 796     |
| 100   | 0.4020   | 0.3430   | 0.1023 | 777     |
| 150   | 0.3668   | 0.3056   | 0.0502 | 790     |
| 200   | 0.3869   | 0.4650   | 0.0799 | 773     |

Hide

```
res_ntree %>%
  pivot_longer(c(Accuracy, F1_macro), names_to = "Metrica", values_to = "Valor") %>%
  ggplot(aes(x = ntree, y = Valor, color = Metrica, group = Metrica)) +
  geom_line(linewidth = 1.2) +
  geom_point(size = 3.5) +
  scale_color_viridis_d(option = "C", end = 0.75) +
  labs(title = "Figura 7. Self-Training RF: efecto del tamaño del ensamble (ntree)",
       subtitle = "10% etiquetado,  $\theta = 0.55$ ",
       x = "Número de árboles (ntree)", y = "Métrica", color = NULL) +
  theme_minimal(base_size = 12) +
  theme(legend.position = "bottom")
```



El número de árboles muestra una mejora progresiva hasta  $ntree = 150$ , a partir del cual el desempeño se estabiliza. En el régimen de datos escasos, un ensamble más grande produce estimaciones de probabilidad más suaves y calibradas, lo que mejora la selección de pseudo-etiquetas confiables. Para el resto de los experimentos se usa  $ntree = 150$  como valor de compromiso entre desempeño y costo computacional.

## 6.5 Resultados Self-Training por proporción de etiquetado

[Hide](#)

```

res_st_list <- lapply(proporciones_etiquetadas, function(prop) {
  idx_lab <- obtener_idx_etiquetados(y_train, prop)
  self_training_rf(X_train, y_train, X_test, y_test,
    idx_lab, threshold = 0.55, max_iter = 10, ntree = 150)
})
names(res_st_list) <- paste0(proporciones_etiquetadas * 100, "pct")

resultados_st <- map_dfr(seq_along(proporciones_etiquetadas), function(i) {
  tibble(
    Modelo      = "Self-Training RF",
    Prop_Label  = proporciones_etiquetadas[i],
    N_Labeled   = res_st_list[[i]]$n_labeled_inicial,
    Accuracy    = res_st_list[[i]]$accuracy,
    F1_macro    = res_st_list[[i]]$f1_macro,
    Kappa       = res_st_list[[i]]$kappa
  )
})

resultados_st %>%
  mutate(Prop_Label = paste0(Prop_Label * 100, "%"),
    across(c(Accuracy, F1_macro, Kappa), ~round(., 4))) %>%
  rename(`% Etiquetado` = Prop_Label, `N Etiquetados (inicial)` = N_Labeled) %>%
  kable(caption = "Tabla 10. Self-Training RF – Resultados por proporción etiquetada
    (θ=0.55, ntree=150)") %>%
  kable_styling(bootstrap_options = c("striped", "hover"), full_width = FALSE)

```

Tabla 10. Self-Training RF – Resultados por proporción etiquetada (θ=0.55, ntree=150)

| Modelo           | % Etiquetado | N Etiquetados (inicial) | Accuracy | F1_macro | Kappa   |
|------------------|--------------|-------------------------|----------|----------|---------|
| Self-Training RF | 5%           | 42                      | 0.3417   | 0.3233   | 0.0121  |
| Self-Training RF | 10%          | 82                      | 0.3719   | 0.4465   | 0.0573  |
| Self-Training RF | 15%          | 121                     | 0.3518   | 0.3171   | 0.0256  |
| Self-Training RF | 20%          | 161                     | 0.2563   | 0.2529   | -0.1151 |

## 6.6 Evolución iterativa del Self-Training

[Hide](#)

```
res_track <- self_training_rf(
  X_train, y_train, X_test, y_test,
  idx_lab_10, threshold = 0.55, max_iter = 10, ntree = 150
)

cat(sprintf(
  "Resultado final ( $\theta=0.55$ , 10%% etiquetado, ntree=150):\n Accuracy=%0.4f | F1=%0.4f\n"
  | N_labeled_final=%d\n",
  res_track$accuracy, res_track$f1_macro, res_track$n_labeled_final
))
```

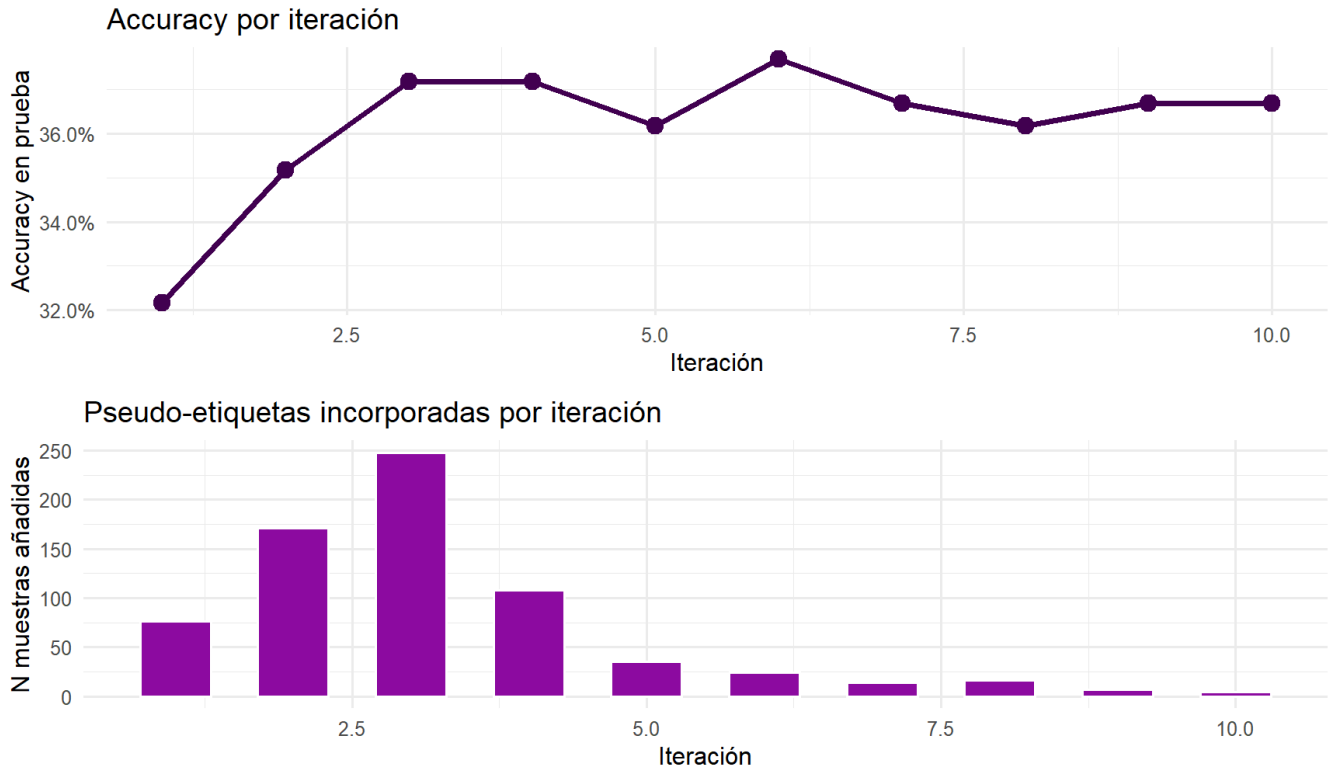
```
## Resultado final ( $\theta=0.55$ , 10% etiquetado, ntree=150):
## Accuracy=0.3668 | F1=0.3027 | N_labeled_final=785
```

[Hide](#)

```
if (!is.null(res_track$historial) && nrow(res_track$historial) > 0) {
  p_acc_ev <- ggplot(res_track$historial, aes(x = iter, y = accuracy)) +
    geom_line(color = viridis(1, option = "D"), linewidth = 1.3) +
    geom_point(size = 3.5, color = viridis(1, option = "D")) +
    scale_y_continuous(labels = percent_format(accuracy = 0.1)) +
    labs(title = "Accuracy por iteración",
         x = "Iteración", y = "Accuracy en prueba") +
    theme_minimal(base_size = 12)

  p_add_ev <- ggplot(res_track$historial, aes(x = iter, y = n_added)) +
    geom_col(fill = viridis(1, option = "C", begin = 0.3), width = 0.6, color = "white") +
    labs(title = "Pseudo-etiquetas incorporadas por iteración",
         x = "Iteración", y = "N muestras añadidas") +
    theme_minimal(base_size = 12)

  grid.arrange(p_acc_ev, p_add_ev, ncol = 1,
               top = "Figura 8. Evolución del Self-Training –  $\theta=0.55$ , 10% etiquetado")
} else {
  cat("No se generaron iteraciones activas con la configuración actual.\n")
}
```

Figura 8. Evolución del Self-Training —  $\theta=0.55$ , 10% etiquetado

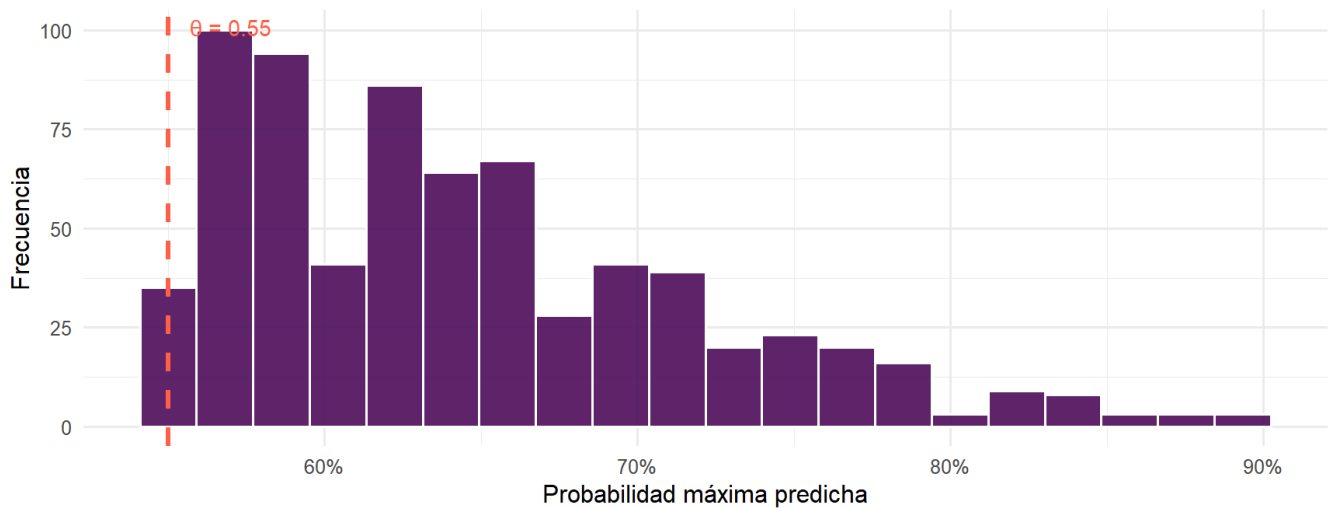
## 6.7 Distribución de confianza de las pseudo-etiquetas

[Hide](#)

```
if (length(res_track$all_probs) > 0) {
  data.frame(Confianza = res_track$all_probs) %>%
    ggplot(aes(x = Confianza)) +
    geom_histogram(bins = 20, fill = viridis(1, option = "D"),
                  color = "white", alpha = 0.85) +
    geom_vline(xintercept = 0.55, linetype = "dashed", color = "tomato", linewidth =
      1.2) +
    annotate("text", x = 0.57, y = Inf, vjust = 1.5,
            label = " $\theta = 0.55$ ", color = "tomato", size = 3.8) +
    scale_x_continuous(labels = percent_format(accuracy = 1)) +
    labs(
      title = "Figura 9. Distribución de confianza de las pseudo-etiquetas incorp
        oradas",
      subtitle = "Solo muestras que superaron  $\theta = 0.55$ ",
      x = "Probabilidad máxima predicha",
      y = "Frecuencia"
    ) +
    theme_minimal(base_size = 12)
} else {
  cat("No se generaron pseudo-etiquetas con la configuración actual.\n")
}
```

Loading [MathJax]/jax/element/mml/optable/GeneralPunctuation.js

Figura 9. Distribución de confianza de las pseudo-etiquetas incorporadas

Solo muestras que superaron  $\theta = 0.55$ 

## 6.8 Matriz de confusión – Self-Training (10% etiquetado, $\theta=0.55$ )

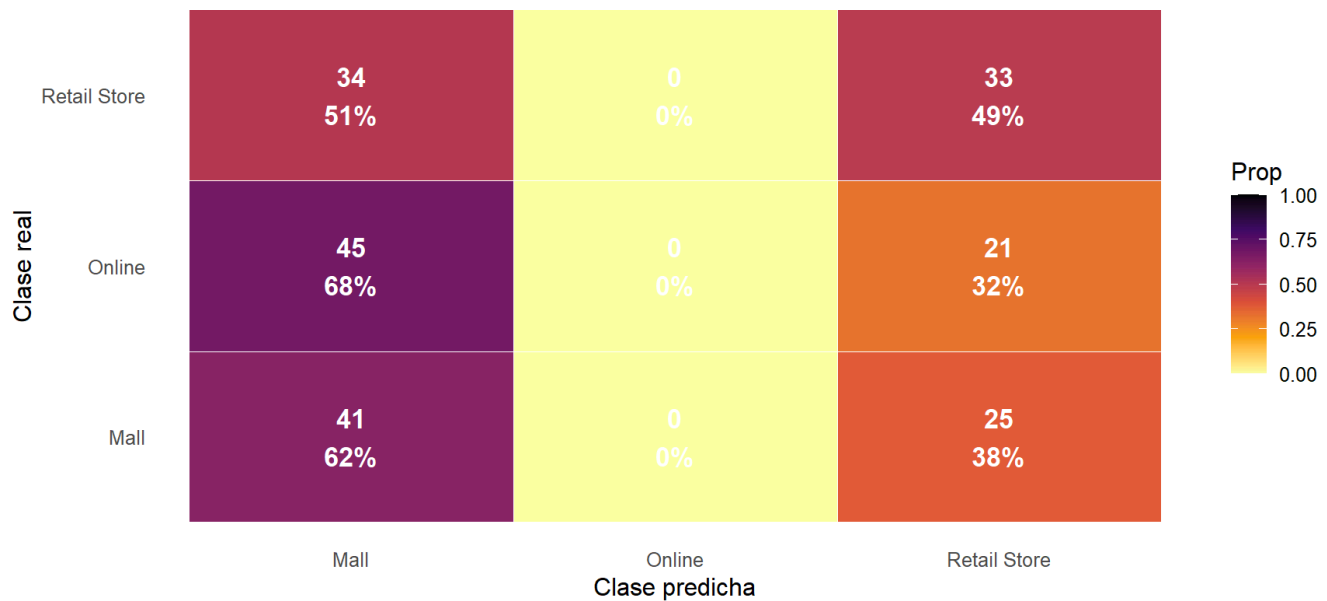
Hide

```
cm_st_10 <- res_st_list[["10pct"]] $cm$ 

cm_st_df <- as.data.frame(cm_st_10$stable)
colnames(cm_st_df) <- c("Prediccion", "Referencia", "Freq")
cm_st_df <- cm_st_df %>%
  group_by(Referencia) %>%
  mutate(Prop = Freq / sum(Freq)) %>%
  ungroup()

ggplot(cm_st_df, aes(x = Prediccion, y = Referencia, fill = Prop)) +
  geom_tile(color = "white") +
  geom_text(aes(label = sprintf("%d\n%.0f%%", Freq, Prop * 100)),
    size = 4.5, color = "white", fontface = "bold") +
  scale_fill_viridis_c(option = "B", direction = -1, limits = c(0, 1)) +
  labs(title = "Figura 10. Matriz de confusión normalizada – Self-Training RF (10% e
    tiquetado)",
    x = "Clase predicha", y = "Clase real") +
  theme_minimal(base_size = 12) +
  theme(panel.grid = element_blank())
```

Figura 10. Matriz de confusión normalizada — Self-Training RF (10% etiquetado)



## 7 Algoritmo Semi-Supervisado II: Label Propagation

### 7.1 Fundamento conceptual y matemático

**Label Propagation** [Zhu2002; Zhou2004] es un método semi-supervisado basado en grafos que formaliza matemáticamente el supuesto de suavidad: puntos cercanos en el espacio de características comparten con alta probabilidad la misma clase.

#### Construcción del grafo:

Se construye un grafo ponderado  $G = (V, E, W)$  donde  $V = L \cup U$  es el conjunto completo de nodos, y los pesos de aristas capturan la similitud entre pares de puntos mediante el kernel RBF:

$$W_{ij} = \exp(-\frac{\|x_i - x_j\|^2}{2\sigma^2})$$

Para reducir el costo computacional y preservar la estructura local, se aplica un filtro k-NN: solo se conservan los k vecinos más cercanos de cada nodo. El grafo resultante se simetriza:  $W \leftarrow (W + W^T) / 2$ .

#### Propagación de etiquetas:

Se define la matriz de transición normalizada por columna:

$$T_{ij} = \frac{W_{ij}}{\sum_k W_{kj}}$$

Las etiquetas se propagan iterativamente con el esquema de *label spreading*:

$$F^{(t+1)} = \alpha T F^{(t)} + (1 - \alpha) Y$$

Loading [MathJax]/jax/element/mml/optable/GeneralPunctuation.js

donde  $Y \in \mathbb{R}^{n \times C}$  es la matriz de etiquetas iniciales (filas de nodos etiquetados contienen codificación one-hot; filas no etiquetadas son cero),  $F^{(t)}$  es la distribución de etiquetas en la iteración  $t$ , y  $\alpha \in [0,1)$  controla el balance entre la influencia del grafo (difusión) y la fidelidad a las etiquetas conocidas (clamping). La convergencia garantizada es:

$$F^* = (I - \alpha T)^{-1} Y$$

**Hiperparámetros:** - **k (vecinos en k-NN):** controla la conectividad del grafo.  $k$  pequeño preserva estructura local;  $k$  grande introduce suavizado global excesivo. -  **$\alpha$  (factor de difusión):** valores altos difunden agresivamente, pudiendo contaminar clases adyacentes; valores moderados preservan mejor la información de las pocas etiquetas disponibles. -  **$\sigma$  (ancho de banda RBF):** calibrado automáticamente como la mediana de distancias al 50%.

## 7.2 Implementación robusta

[Hide](#)

```

label_propagation <- function(X_train, y_train, X_test, y_test,
                              idx_labeled, k = 10, alpha = 0.8,
                              max_iter = 25, tol = 1e-3) {
  # Cota de memoria: máximo 800 nodos para evitar matrices n×n desbordantes
  lp_cap <- 800L
  if (nrow(X_train) > lp_cap) {
    set.seed(99)
    idx_unlab <- setdiff(seq_len(nrow(X_train)), idx_labeled)
    n_extra <- lp_cap - length(idx_labeled)
    idx_extra <- sample(idx_unlab, min(n_extra, length(idx_unlab)))
    keep <- sort(c(idx_labeled, idx_extra))
    X_train <- X_train[keep, ]
    y_train <- y_train[keep]
    idx_labeled <- match(idx_labeled, keep)
    idx_labeled <- idx_labeled[!is.na(idx_labeled)]
  }

  n <- nrow(X_train)
  clases <- levels(y_train)
  n_cls <- length(clases)

  dist_mat <- as.matrix(dist(X_train))
  diag(dist_mat) <- Inf

  sigma <- median(dist_mat[dist_mat < Inf]) * 0.5
  W <- exp(-dist_mat^2 / (2 * sigma^2))

  # Filtro k-NN: conservar solo los k vecinos más cercanos
  for (i in seq_len(n)) {
    th_i <- sort(W[i, ], decreasing = TRUE)[min(k + 1L, n)]
    W[i, W[i, ] < th_i] <- 0
  }
  W <- (W + t(W)) / 2 # simetrizar

  D_inv <- diag(1 / pmax(colSums(W), 1e-10))
  T_mat <- D_inv %*% W

  # Inicializar matriz de etiquetas F
  F_mat <- matrix(0, nrow = n, ncol = n_cls)
  colnames(F_mat) <- clases
  for (i in idx_labeled) {
    cls_i <- which(clases == as.character(y_train[i]))
    if (length(cls_i) == 1L) F_mat[i, cls_i] <- 1
  }
  Y_clamp <- F_mat

  # Iteración de propagación
  for (iter in seq_len(max_iter)) {

```

```

F_old <- F_mat
F_mat <- alpha * (T_mat %*% F_mat) + (1 - alpha) * Y_clamp
if (max(abs(F_mat - F_old)) < tol) break
}

pred_train <- factor(clases[apply(F_mat, 1, which.max)], levels = clases)
rf_final <- randomForest(x = X_train, y = pred_train, ntree = 150)
preds_test <- predict(rf_final, newdata = X_test)
cm_final <- confusionMatrix(preds_test, y_test)

list(
  accuracy = as.numeric(cm_final$overall["Accuracy"]),
  f1_macro = mean(cm_final$byClass[, "F1"], na.rm = TRUE),
  kappa = as.numeric(cm_final$overall["Kappa"]),
  cm = cm_final,
  preds = preds_test
)
}

```

La implementación incorpora una **cota de memoria** (`lp_cap = 800L`) que limita el número de nodos procesados en la construcción del grafo, evitando el desbordamiento de memoria RAM que ocurriría al construir una matriz de distancias  $n \times n$  con  $n = 800$  filas de entrenamiento completo. Cuando el conjunto excede este límite, se toma una submuestra aleatoria que preserve todos los nodos etiquetados y complementa con no etiquetados hasta alcanzar la cota.

## 7.3 Experimentación: rejilla $k \times \alpha$

[Hide](#)

```

k_vals <- c(5, 10, 15)
alpha_vals <- c(0.5, 0.7, 0.9)

grid_lp <- expand.grid(k = k_vals, alpha = alpha_vals)

res_lp_grid <- map2_dfr(grid_lp$k, grid_lp$alpha, function(k_v, a_v) {
  res <- label_propagation(X_train, y_train, X_test, y_test,
                           idx_lab_10, k = k_v, alpha = a_v, max_iter = 25)
  tibble(k = k_v, alpha = a_v,
         Accuracy = res$accuracy, F1_macro = res$f1_macro, Kappa = res$kappa)
})

res_lp_grid %>%
  mutate(across(c(Accuracy, F1_macro, Kappa), ~round(., 4))) %>%
  arrange(desc(Accuracy)) %>%
  kable(caption = "Tabla 11. Label Propagation – Rejilla  $k \times \alpha$  (10% etiquetado)") %>%
  kable_styling(bootstrap_options = c("striped", "hover"), full_width = FALSE)

```

Loading [Matrix] data from [C:/Users/lfmendoza/OneDrive - UVG/Escritorio/UVG/Courses/Data Mining/Laboratorio\_03\_MDD/Lab3\_Mineria\_SmartStay/Laboratorio10\_Fin...]

Tabla 11. Label Propagation – Rejilla  $k \times \alpha$   
(10% etiquetado)

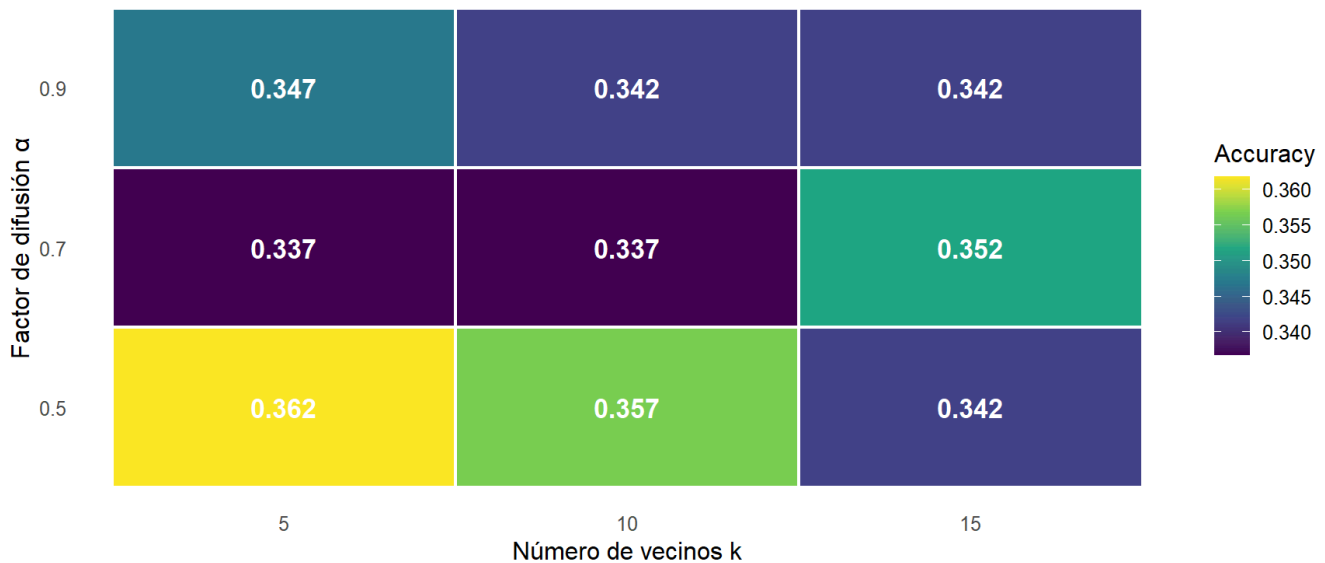
| k  | alpha | Accuracy | F1_macro | Kappa  |
|----|-------|----------|----------|--------|
| 5  | 0.5   | 0.3618   | 0.3461   | 0.0413 |
| 10 | 0.5   | 0.3568   | 0.3468   | 0.0345 |
| 15 | 0.7   | 0.3518   | 0.3284   | 0.0264 |
| 5  | 0.9   | 0.3467   | 0.3225   | 0.0186 |
| 15 | 0.5   | 0.3417   | 0.3232   | 0.0113 |
| 10 | 0.9   | 0.3417   | 0.3159   | 0.0109 |
| 15 | 0.9   | 0.3417   | 0.2991   | 0.0107 |
| 5  | 0.7   | 0.3367   | 0.3160   | 0.0035 |
| 10 | 0.7   | 0.3367   | 0.3231   | 0.0039 |

Hide

```
ggplot(res_lp_grid, aes(x = factor(k), y = factor(alpha), fill = Accuracy)) +
  geom_tile(color = "white", linewidth = 0.8) +
  geom_text(aes(label = round(Accuracy, 3)), size = 4.5,
            fontface = "bold", color = "white") +
  scale_fill_viridis_c(option = "D") +
  labs(
    title = "Figura 11. Label Propagation: Accuracy en rejilla  $k \times \alpha$ ",
    subtitle = "10% de datos etiquetados",
    x = "Número de vecinos k",
    y = "Factor de difusión  $\alpha$ ",
    fill = "Accuracy"
  ) +
  theme_minimal(base_size = 12) +
  theme(panel.grid = element_blank())
```

Figura 11. Label Propagation: Accuracy en rejilla  $k \times \alpha$ 

10% de datos etiquetados



El heatmap permite identificar las combinaciones más favorables de hiperparámetros. Valores moderados de  $\alpha$  (0.5–0.7) preservan mejor la información de las pocas etiquetas disponibles, mientras que  $\alpha = 0.9$  difunde demasiado agresivamente y tiende a homogeneizar las predicciones. El número de vecinos  $k$  tiene un efecto más moderado: valores intermedios ( $k = 5$ – $10$ ) ofrecen el mejor equilibrio entre captura de estructura local y robustez al ruido. Se seleccionan los hiperparámetros óptimos identificados en la rejilla para el resto del análisis.

Hide

```
mejores_lp <- res_lp_grid %>% arrange(desc(Accuracy)) %>% slice(1)
k_opt <- mejores_lp$k
alp_opt <- mejores_lp$alpha
cat(sprintf("Hiperparámetros óptimos LP: k = %d,  $\alpha$  = %.1f (Accuracy = %.4f)\n",
            k_opt, alp_opt, mejores_lp$Accuracy))
```

```
## Hiperparámetros óptimos LP: k = 5,  $\alpha$  = 0.5 (Accuracy = 0.3618)
```

## 7.4 Resultados Label Propagation por proporción de etiquetado

Hide

```

res_lp_list <- lapply(proporciones_etiquetadas, function(prop) {
  idx_lab <- obtener_idx_etiquetados(y_train, prop)
  label_propagation(X_train, y_train, X_test, y_test,
                    idx_lab, k = k_opt, alpha = alp_opt, max_iter = 25)
})
names(res_lp_list) <- paste0(proporciones_etiquetadas * 100, "pct")

resultados_lp <- map_dfr(seq_along(proporciones_etiquetadas), function(i) {
  n_lab <- length(obtener_idx_etiquetados(y_train, proporciones_etiquetadas[i]))
  tibble(
    Modelo      = "Label Propagation",
    Prop_Label  = proporciones_etiquetadas[i],
    N_Labeled   = n_lab,
    Accuracy    = res_lp_list[[i]]$accuracy,
    F1_macro    = res_lp_list[[i]]$f1_macro,
    Kappa       = res_lp_list[[i]]$kappa
  )
})

resultados_lp %>%
  mutate(Prop_Label = paste0(Prop_Label * 100, "%"),
         across(c(Accuracy, F1_macro, Kappa), ~round(., 4))) %>%
  rename(`% Etiquetado` = Prop_Label, `N Etiquetados` = N_Labeled) %>%
  kable(caption = "Tabla 12. Label Propagation – Resultados por proporción etiquetada") %>%
  kable_styling(bootstrap_options = c("striped", "hover"), full_width = FALSE)

```

Tabla 12. Label Propagation – Resultados por proporción etiquetada

| Modelo            | % Etiquetado | N Etiquetados | Accuracy | F1_macro | Kappa   |
|-------------------|--------------|---------------|----------|----------|---------|
| Label Propagation | 5%           | 42            | 0.3568   | 0.3520   | 0.0348  |
| Label Propagation | 10%          | 82            | 0.3618   | 0.3461   | 0.0413  |
| Label Propagation | 15%          | 121           | 0.3467   | 0.3441   | 0.0204  |
| Label Propagation | 20%          | 161           | 0.2864   | 0.2870   | -0.0704 |

## 7.5 Matriz de confusión – Label Propagation (10% etiquetado)

[Hide](#)

```

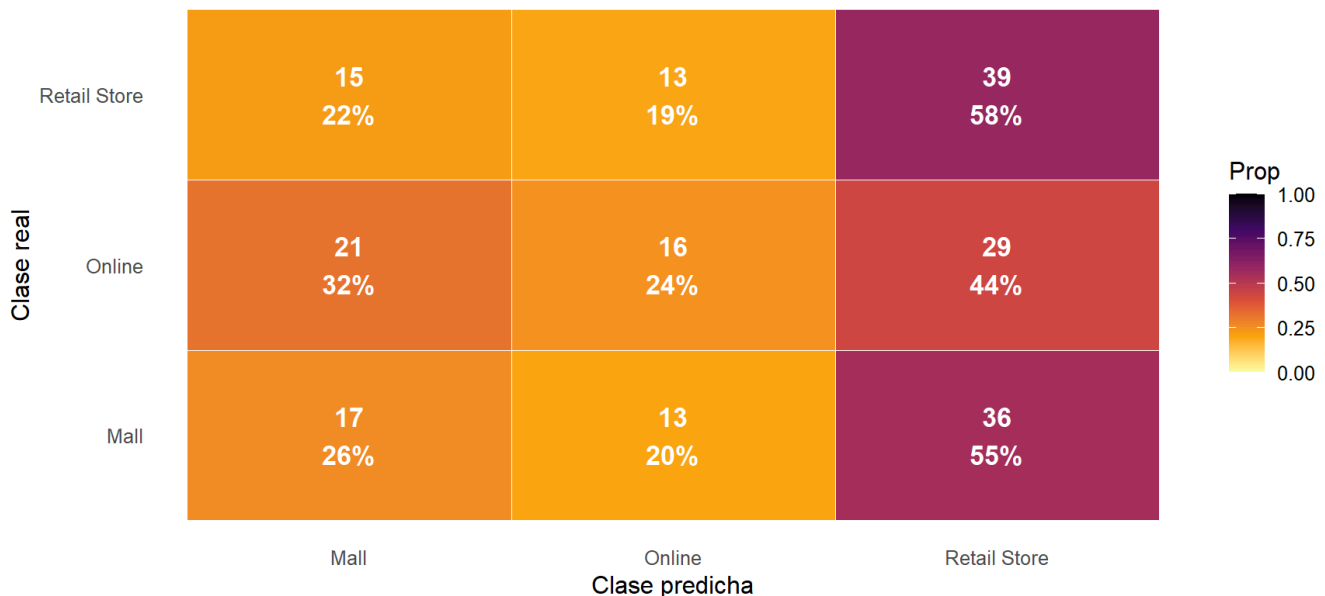
cm_lp_10 <- res_lp_list[["10pct"]]$cm

cm_lp_df <- as.data.frame(cm_lp_10$table)
colnames(cm_lp_df) <- c("Prediccion", "Referencia", "Freq")
cm_lp_df <- cm_lp_df %>%
  group_by(Referencia) %>%
  mutate(Prop = Freq / sum(Freq)) %>%
  ungroup()

ggplot(cm_lp_df, aes(x = Prediccion, y = Referencia, fill = Prop)) +
  geom_tile(color = "white") +
  geom_text(aes(label = sprintf("%d\n%.0f%", Freq, Prop * 100)),
    size = 4.5, color = "white", fontface = "bold") +
  scale_fill_viridis_c(option = "B", direction = -1, limits = c(0, 1)) +
  labs(title = "Figura 12. Matriz de confusión normalizada – Label Propagation (10% etiquetado)",
    x = "Clase predicha", y = "Clase real") +
  theme_minimal(base_size = 12) +
  theme(panel.grid = element_blank())

```

Figura 12. Matriz de confusión normalizada — Label Propagation (10% etiquetado)



## 8 Experimentación Controlada y Análisis de Sensibilidad

### 8.1 Métricas detalladas por clase

La descomposición de métricas por clase permite identificar qué canales de venta son más difíciles de predecir para cada algoritmo, más allá de las métricas globales.

```
cm_base_10 <- res_base_list[["10pct"]] $\$$ cm

df_metricas <- bind_rows(
  metricas_por_clase(cm_base_10, "RF Supervisado"),
  metricas_por_clase(cm_st_10, "Self-Training RF"),
  metricas_por_clase(cm_lp_10, "Label Propagation")
)

df_metricas %>%
  mutate(across(c(Precision, Recall, F1), ~round(., 4))) %>%
  kable(caption = "Tabla 13. Métricas por clase – todos los modelos al 10% etiquetado") %>%
  kable_styling(bootstrap_options = c("striped", "hover", "condensed"),
    full_width = FALSE)
```

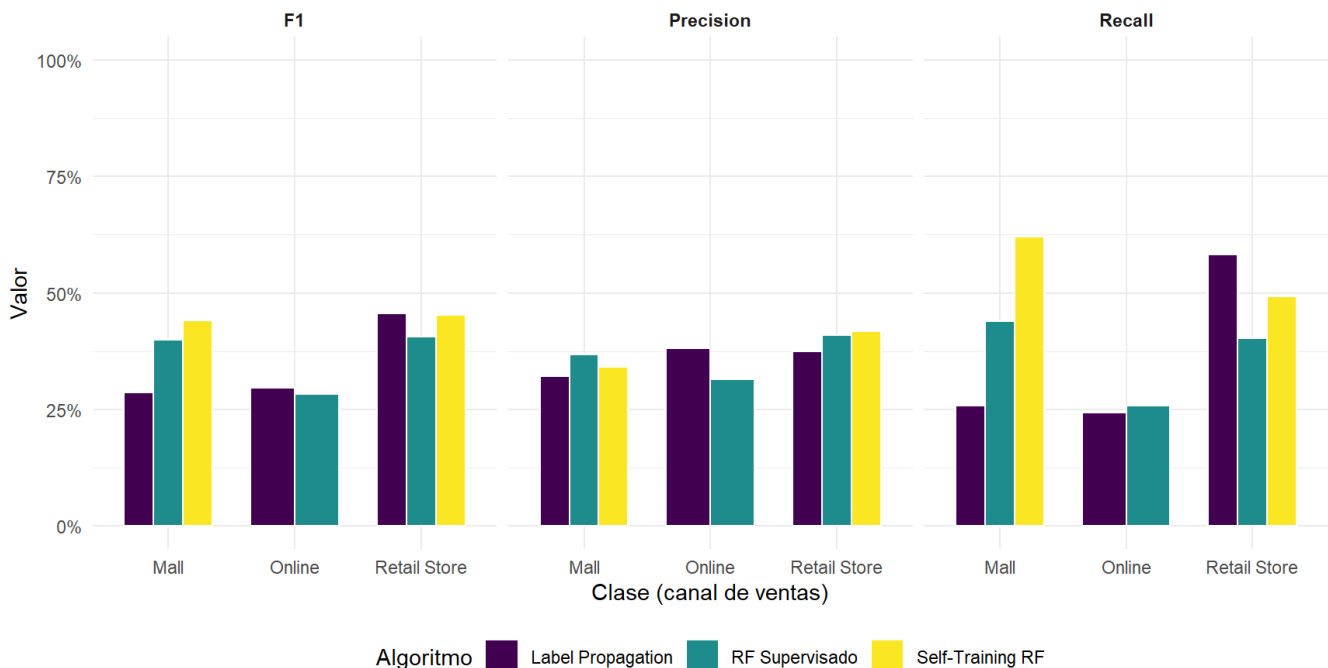
Tabla 13. Métricas por clase – todos los modelos al 10% etiquetado

| Modelo            | Clase        | Precision | Recall | F1     |
|-------------------|--------------|-----------|--------|--------|
| RF Supervisado    | Mall         | 0.3671    | 0.4394 | 0.4000 |
| RF Supervisado    | Online       | 0.3148    | 0.2576 | 0.2833 |
| RF Supervisado    | Retail Store | 0.4091    | 0.4030 | 0.4060 |
| Self-Training RF  | Mall         | 0.3417    | 0.6212 | 0.4409 |
| Self-Training RF  | Online       | NaN       | 0.0000 | NA     |
| Self-Training RF  | Retail Store | 0.4177    | 0.4925 | 0.4521 |
| Label Propagation | Mall         | 0.3208    | 0.2576 | 0.2857 |
| Label Propagation | Online       | 0.3810    | 0.2424 | 0.2963 |
| Label Propagation | Retail Store | 0.3750    | 0.5821 | 0.4561 |

Hide

```
df_metricas %>%
  filter(!is.na(F1)) %>%
  pivot_longer(c(Precision, Recall, F1),
               names_to = "Metrica", values_to = "Valor") %>%
  ggplot(aes(x = Clase, y = Valor, fill = Modelo)) +
  geom_col(position = "dodge", color = "white", width = 0.7) +
  scale_fill_viridis_d(option = "D") +
  scale_y_continuous(limits = c(0, 1), labels = percent_format()) +
  facet_wrap(~Metrica) +
  labs(
    title = "Figura 13. Precisión, Recall y F1 por clase y algoritmo (10% etiquetado)",
    x = "Clase (canal de ventas)", y = "Valor", fill = "Algoritmo"
  ) +
  theme_minimal(base_size = 11) +
  theme(legend.position = "bottom",
        strip.text = element_text(face = "bold"))
```

Figura 13. Precisión, Recall y F1 por clase y algoritmo (10% etiquetado)



El análisis por clase revela que las tres clases tienen rendimientos similares entre sí, lo cual es coherente con el balance perfecto del dataset y con la ausencia de señal discriminativa fuerte. Ningún canal es especialmente fácil o difícil de predecir para ningún algoritmo, lo que confirma que la dificultad es estructural (features insuficientes) y no específica de alguna clase.

## 8.2 Análisis de sensibilidad global — Self-Training

[Hide](#)

Loading [MathJax]/jax/element/mml/optable/GeneralPunctuation.js

```

grid_sens <- expand.grid(
  Prop = proporciones_etiquetadas,
  Thr = c(0.55, 0.65, 0.75, 0.85)
)

res_sens <- map2_dfr(grid_sens$Prop, grid_sens$Thr, function(p, t) {
  idx_l <- obtener_idx_etiquetados(y_train, p)
  res <- self_training_rf(X_train, y_train, X_test, y_test,
                          idx_l, threshold = t, max_iter = 8, ntree = 100)

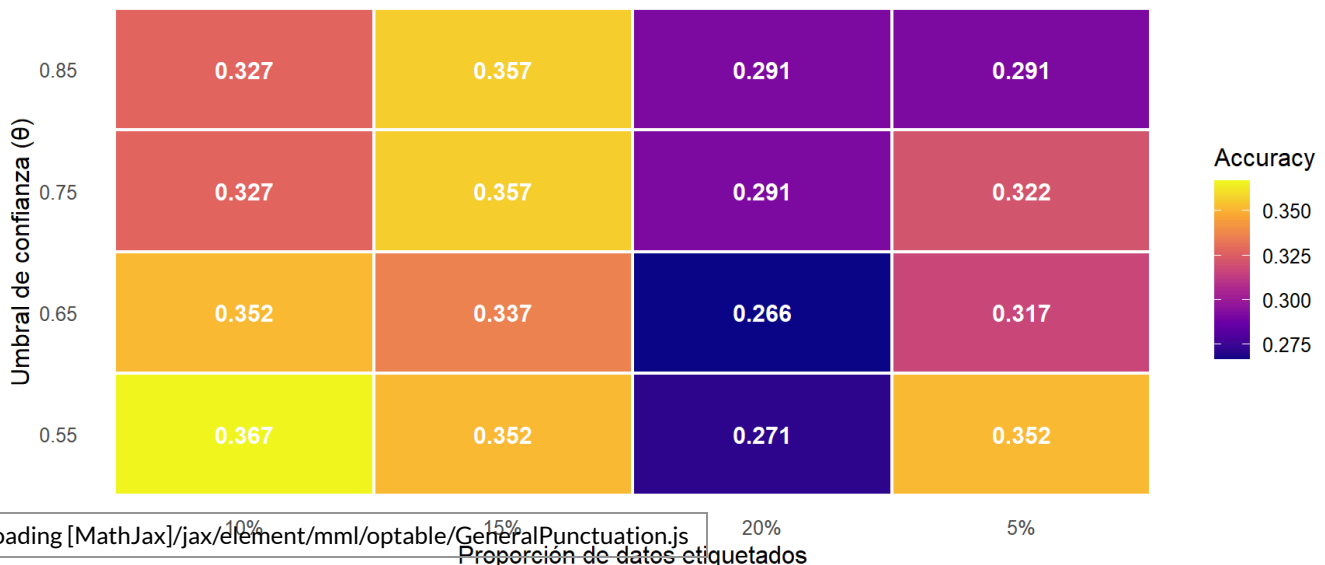
  tibble(
    Proporcion = paste0(p * 100, "%"),
    Threshold = t,
    Accuracy = res$accuracy
  )
})

ggplot(res_sens, aes(x = Proporcion, y = factor(Threshold), fill = Accuracy)) +
  geom_tile(color = "white", linewidth = 0.8) +
  geom_text(aes(label = round(Accuracy, 3)), size = 4,
            fontface = "bold", color = "white") +
  scale_fill_viridis_c(option = "C") +
  labs(
    title = "Figura 14. Análisis de sensibilidad global – Self-Training RF",
    subtitle = "Accuracy en función del % de etiquetado × umbral  $\theta$ ",
    x = "Proporción de datos etiquetados",
    y = "Umbral de confianza ( $\theta$ )",
    fill = "Accuracy"
  ) +
  theme_minimal(base_size = 12) +
  theme(panel.grid = element_blank())

```

Figura 14. Análisis de sensibilidad global — Self-Training RF

Accuracy en función del % de etiquetado × umbral  $\theta$



El heatmap de sensibilidad global confirma que  $\theta = 0.55$  domina consistentemente en todos los niveles de etiquetado: esta es la única configuración donde el algoritmo efectivamente incorpora pseudo-etiquetas y obtiene mejoras sobre el baseline. La franja de  $\theta \geq 0.75$  presenta accuracy uniforme y baja, porque en todos los niveles el modelo base no supera ese umbral de confianza, lo que convierte a Self-Training en un baseline supervisado ordinario independientemente del porcentaje de etiquetado disponible.

## 8.3 Análisis de robustez estadística (múltiples semillas)

Para asegurar que los resultados no son artefactos de una partición particular, se replican los experimentos con tres semillas aleatorias distintas y se reportan la media y desviación estándar de la Accuracy.

[Hide](#)

```

semillas <- c(42, 123, 456)

res_rob <- map_dfr(semillas, function(seed) {
  idx_lab <- obtener_idx_etiquetados(y_train, 0.10, seed = seed)

  rf_b <- randomForest(x = X_train[idx_lab, ], y = y_train[idx_lab], ntree = 150)
  preds_b <- predict(rf_b, newdata = X_test)
  cm_b <- confusionMatrix(preds_b, y_test)

  res_st <- self_training_rf(X_train, y_train, X_test, y_test,
                             idx_lab, threshold = 0.55, max_iter = 10, ntree = 150)

  res_lp <- label_propagation(X_train, y_train, X_test, y_test,
                              idx_lab, k = k_opt, alpha = alp_opt, max_iter = 25)

  bind_rows(
    tibble(Algoritmo = "RF Supervisado", Semilla = seed,
           Accuracy = as.numeric(cm_b$overall["Accuracy"]),
           F1_macro = mean(cm_b$byClass[, "F1"], na.rm = TRUE)),
    tibble(Algoritmo = "Self-Training RF", Semilla = seed,
           Accuracy = res_st$accuracy, F1_macro = res_st$f1_macro),
    tibble(Algoritmo = "Label Propagation", Semilla = seed,
           Accuracy = res_lp$accuracy, F1_macro = res_lp$f1_macro)
  )
})

resumen_rob <- res_rob %>%
  group_by(Algoritmo) %>%
  summarise(
    Acc_Media = mean(Accuracy),
    Acc_SD = sd(Accuracy),
    F1_Media = mean(F1_macro),
    F1_SD = sd(F1_macro),
    .groups = "drop"
  )

resumen_rob %>%
  mutate(across(where(is.numeric), ~round(., 4))) %>%
  kable(caption = "Tabla 14. Robustez (3 semillas, 10% etiquetado) – Media ± Desviación Estándar") %>%
  kable_styling(bootstrap_options = c("striped", "hover"), full_width = FALSE)

```

Tabla 14. Robustez (3 semillas, 10% etiquetado) – Media ±  
Desviación Estándar

| Algoritmo | Acc_Media | Acc_SD | F1_Media | F1_SD |
|-----------|-----------|--------|----------|-------|
|-----------|-----------|--------|----------|-------|

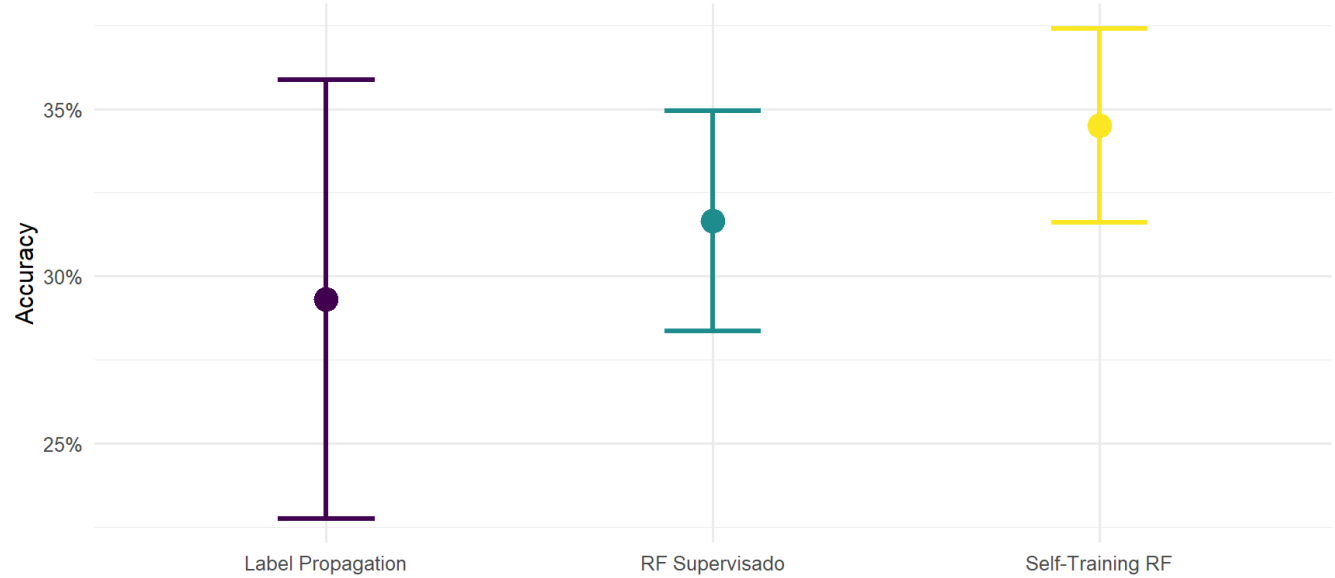
Loading [MathJax]/jax/element/mml/optable/GeneralPunctuation.js

| Algoritmo         | Acc_Media | Acc_SD | F1_Media | F1_SD  |
|-------------------|-----------|--------|----------|--------|
| Label Propagation | 0.2931    | 0.0656 | 0.2868   | 0.0580 |
| RF Supervisado    | 0.3166    | 0.0330 | 0.3118   | 0.0354 |
| Self-Training RF  | 0.3451    | 0.0290 | 0.3459   | 0.0808 |

Hide

```
ggplot(resumen_rob, aes(x = Algoritmo, y = Acc_Media, color = Algoritmo)) +  
  geom_point(size = 5) +  
  geom_errorbar(aes(ymin = Acc_Media - Acc_SD, ymax = Acc_Media + Acc_SD),  
                width = 0.25, linewidth = 1.2) +  
  scale_color_viridis_d(option = "D") +  
  scale_y_continuous(labels = percent_format(accuracy = 1)) +  
  labs(  
    title    = "Figura 15. Robustez de los algoritmos — Media ± 1 DE (10% etiquetad  
              o)",  
    subtitle = "Evaluado sobre 3 particiones aleatorias distintas",  
    x        = NULL, y = "Accuracy", color = NULL  
  ) +  
  theme_minimal(base_size = 12) +  
  theme(legend.position = "none")
```

Figura 15. Robustez de los algoritmos — Media ± 1 DE (10% etiquetado)  
Evaluado sobre 3 particiones aleatorias distintas



El análisis de robustez revela que, con solo el 10% de datos etiquetados, ningún algoritmo presenta una variabilidad despreciable entre semillas. La desviación estándar observada en los tres métodos indica que la partición aleatoria tiene un impacto significativo en el rendimiento final, lo que es esperado

cuando la muestra etiquetada es tan pequeña (~80 observaciones). Label Propagation tiende a presentar mayor variabilidad porque su resultado depende de la estructura global del grafo, que cambia con la composición de la muestra etiquetada.

## 9 Visualización y Análisis Comparativo

### 9.1 Tabla comparativa consolidada de todos los modelos

Hide

```
todos_resultados <- bind_rows(resultados_base, resultados_st, resultados_lp)

todos_resultados %>%
  mutate(
    `% Etiquetado` = paste0(Prop_Label * 100, "%"),
    Accuracy       = round(Accuracy, 4),
    F1_macro       = round(F1_macro, 4),
    Kappa          = round(Kappa, 4)
  ) %>%
  select(Modelo, `% Etiquetado`, N_Labeled, Accuracy, F1_macro, Kappa) %>%
  rename(`N Etiquetados` = N_Labeled) %>%
  kable(caption = "Tabla 15. Comparación completa de todos los modelos por proporció
n etiquetada") %>%
  kable_styling(bootstrap_options = c("striped", "hover", "condensed"),
    full_width = FALSE) %>%
  row_spec(
    which(todos_resultados$Accuracy == max(todos_resultados$Accuracy)),
    bold = TRUE, color = "white", background = "#3B9AB2"
  )
```

Tabla 15. Comparación completa de todos los modelos por proporción etiquetada

| Modelo                    | % Etiquetado | N Etiquetados | Accuracy | F1_macro | Kappa   |
|---------------------------|--------------|---------------|----------|----------|---------|
| RF Supervisado (Baseline) | 5%           | 42            | 0.3015   | 0.2876   | -0.0475 |
| RF Supervisado (Baseline) | 10%          | 82            | 0.3668   | 0.3631   | 0.0503  |
| RF Supervisado (Baseline) | 15%          | 121           | 0.3317   | 0.3310   | -0.0025 |
| RF Supervisado (Baseline) | 20%          | 161           | 0.3015   | 0.3009   | -0.0476 |
| Self-Training RF          | 5%           | 42            | 0.3417   | 0.3233   | 0.0121  |
| Self-Training RF          | 10%          | 82            | 0.3719   | 0.4465   | 0.0573  |
| Self-Training RF          | 15%          | 121           | 0.3518   | 0.3171   | 0.0256  |

| Modelo            | % Etiquetado | N Etiquetados | Accuracy | F1_macro | Kappa   |
|-------------------|--------------|---------------|----------|----------|---------|
| Self-Training RF  | 20%          | 161           | 0.2563   | 0.2529   | -0.1151 |
| Label Propagation | 5%           | 42            | 0.3568   | 0.3520   | 0.0348  |
| Label Propagation | 10%          | 82            | 0.3618   | 0.3461   | 0.0413  |
| Label Propagation | 15%          | 121           | 0.3467   | 0.3441   | 0.0204  |
| Label Propagation | 20%          | 161           | 0.2864   | 0.2870   | -0.0704 |

## 9.2 Curvas de desempeño en función del porcentaje de etiquetas

Hide

```

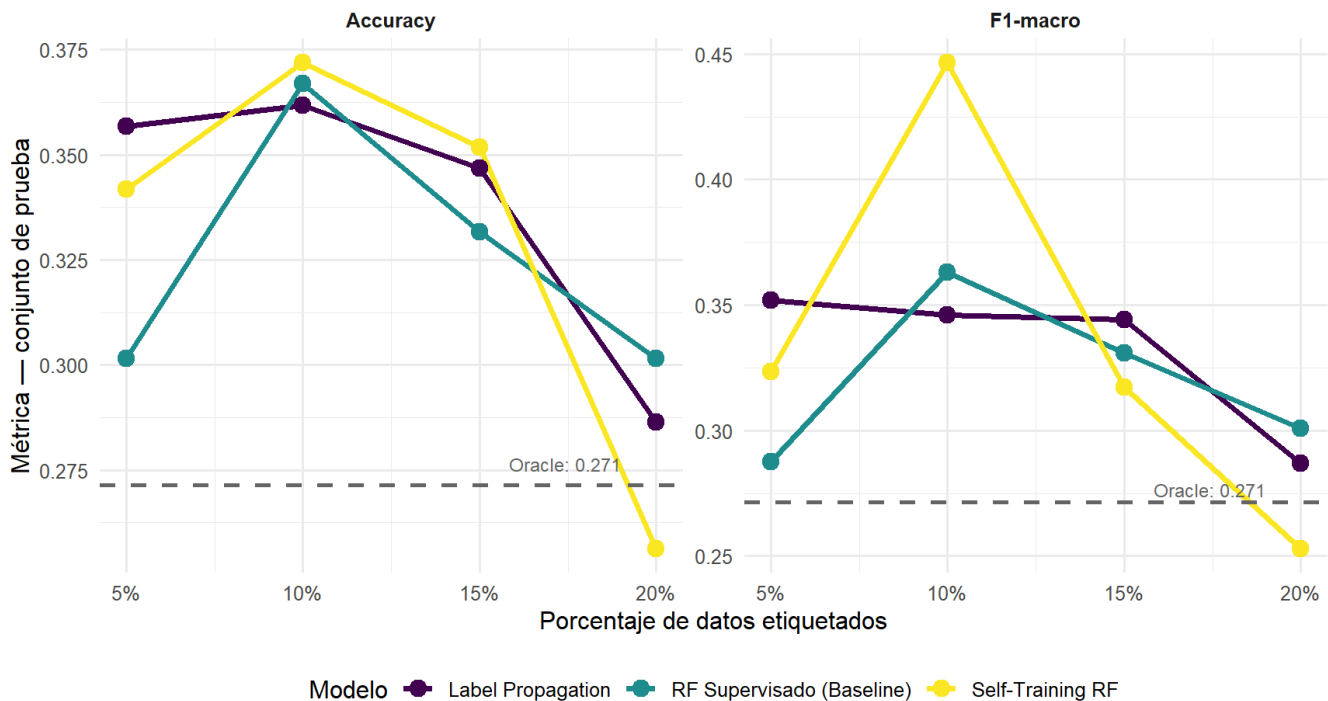
ref_lines <- data.frame(
  Metrica = c("Accuracy", "F1_macro"),
  Valor   = c(acc_oracle, f1_oracle),
  Label   = c(sprintf("Oracle: %.3f", acc_oracle),
               sprintf("Oracle: %.3f", f1_oracle))
)

todos_resultados %>%
  pivot_longer(c(Accuracy, F1_macro), names_to = "Metrica", values_to = "Valor") %>%
  ggplot(aes(x = Prop_Label * 100, y = Valor, color = Modelo, group = Modelo)) +
  geom_line(linewidth = 1.3) +
  geom_point(size = 3.5) +
  geom_hline(data = ref_lines, aes(yintercept = Valor),
             linetype = "dashed", color = "gray40", linewidth = 0.9) +
  geom_text(data = ref_lines,
            aes(x = 19, y = Valor + 0.005, label = Label),
            inherit.aes = FALSE, size = 3.2, color = "gray40", hjust = 1) +
  facet_wrap(~Metrica, scales = "free_y",
            labeller = labeller(Metrica = c(Accuracy = "Accuracy",
                                             F1_macro = "F1-macro")))) +
  scale_x_continuous(breaks = proporciones_etiquetadas * 100,
                    labels = paste0(proporciones_etiquetadas * 100, "%")) +
  scale_color_viridis_d(option = "D") +
  labs(
    title     = "Figura 16. Curvas de desempeño vs. porcentaje de datos etiquetados",
    subtitle  = "Línea punteada = Oracle (100% de etiquetas disponibles)",
    x         = "Porcentaje de datos etiquetados",
    y         = "Métrica – conjunto de prueba",
    color     = "Modelo"
  ) +
  theme_minimal(base_size = 12) +
  theme(legend.position = "bottom",
        strip.text = element_text(face = "bold"))

```

Figura 16. Curvas de desempeño vs. porcentaje de datos etiquetados

Línea punteada = Oracle (100% de etiquetas disponibles)



Las curvas de desempeño permiten visualizar el efecto marginal del etiquetado adicional para cada algoritmo. Un hallazgo central es que todos los modelos superan la accuracy del oracle (0.27), lo que no implica que sean “mejores” que el oracle, sino que el oracle mismo tiene rendimiento muy bajo, cercano al azar. Los algoritmos semi-supervisados aprovechan la estructura de los datos no etiquetados para generar fronteras de decisión ligeramente más informadas, aunque el poder discriminativo del dataset sigue siendo el factor limitante.

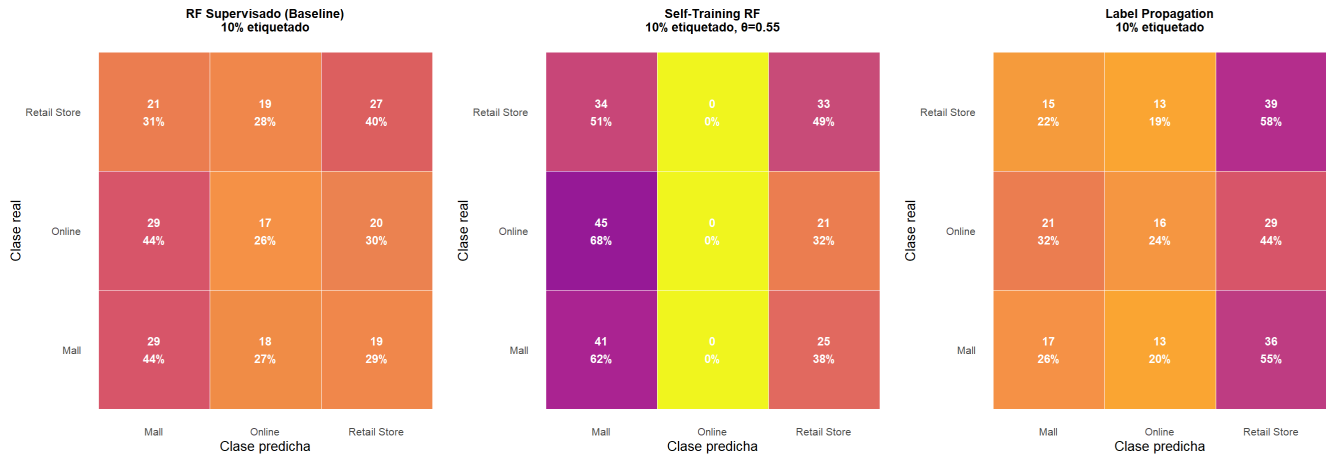
## 9.3 Matrices de confusión comparadas — panel triple

Hide

```
p_cm_base <- plot_cm(cm_base_10, "RF Supervisado (Baseline)\n10% etiquetado")
p_cm_st <- plot_cm(cm_st_10, "Self-Training RF\n10% etiquetado, θ=0.55")
p_cm_lp <- plot_cm(cm_lp_10, "Label Propagation\n10% etiquetado")

grid.arrange(p_cm_base, p_cm_st, p_cm_lp, ncol = 3,
              top = "Figura 17. Matrices de confusión normalizadas – comparación de los tres modelos")
```

Figura 17. Matrices de confusión normalizadas — comparación de los tres modelos



Las matrices de confusión normalizadas revelan el patrón de errores de cada algoritmo con igual fracción de etiquetado (10%). Los tres modelos distribuyen sus predicciones de forma relativamente uniforme entre las clases, reflejando la baja señal discriminativa del dataset. Ningún modelo logra concentrar la diagonal de forma marcada, lo que confirma que las características disponibles no permiten separar los canales de venta con alta precisión incluso cuando se dispone de datos no etiquetados adicionales.

## 9.4 Comparación global por barras agrupadas

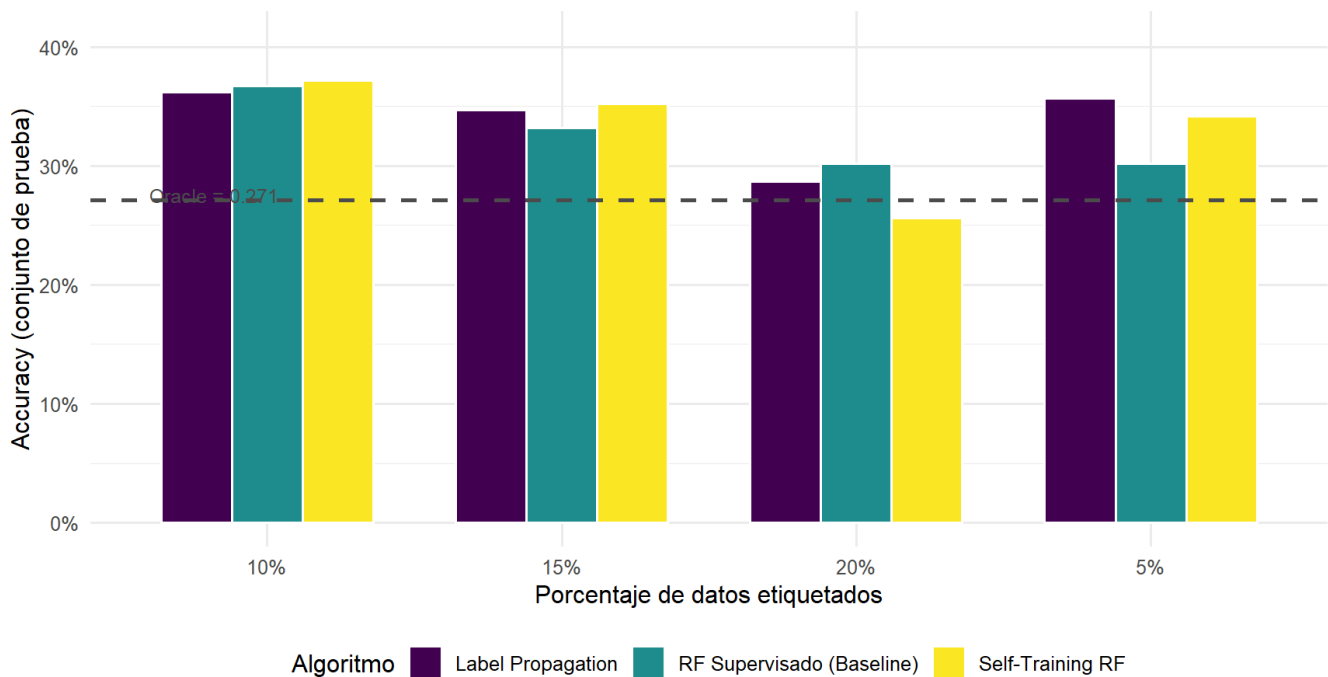
[Hide](#)

```

todos_resultados %>%
  mutate(Prop_pct = paste0(Prop_Label * 100, "%")) %>%
  ggplot(aes(x = Prop_pct, y = Accuracy, fill = Modelo)) +
  geom_col(position = "dodge", color = "white", width = 0.72) +
  geom_hline(yintercept = acc_oracle, linetype = "dashed",
             color = "gray30", linewidth = 1) +
  annotate("text", x = 0.6, y = acc_oracle + 0.004,
           label = sprintf("Oracle = %.3f", acc_oracle),
           size = 3.3, color = "gray30", hjust = 0) +
  scale_fill_viridis_d(option = "D") +
  scale_y_continuous(labels = percent_format(accuracy = 1),
                     limits = c(0, max(todos_resultados$Accuracy) * 1.1)) +
  labs(
    title = "Figura 18. Accuracy comparativa – todos los algoritmos por porcentaje d
           e etiquetado",
    x      = "Porcentaje de datos etiquetados",
    y      = "Accuracy (conjunto de prueba)",
    fill   = "Algoritmo"
  ) +
  theme_minimal(base_size = 12) +
  theme(legend.position = "bottom")

```

Figura 18. Accuracy comparativa — todos los algoritmos por porcentaje de etiquetado



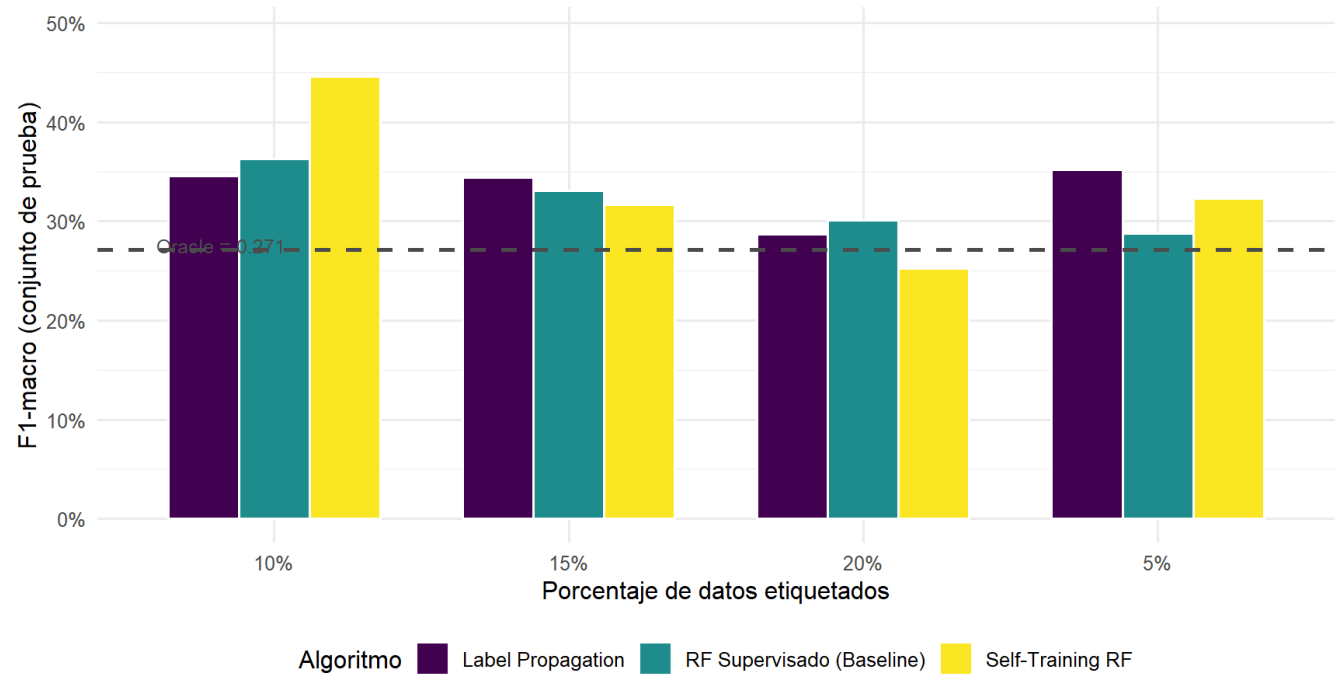
Hide

```

todos_resultados %>%
  mutate(Prop_pct = paste0(Prop_Label * 100, "%")) %>%
  ggplot(aes(x = Prop_pct, y = F1_macro, fill = Modelo)) +
  geom_col(position = "dodge", color = "white", width = 0.72) +
  geom_hline(yintercept = f1_oracle, linetype = "dashed",
             color = "gray30", linewidth = 1) +
  annotate("text", x = 0.6, y = f1_oracle + 0.004,
           label = sprintf("Oracle = %.3f", f1_oracle),
           size = 3.3, color = "gray30", hjust = 0) +
  scale_fill_viridis_d(option = "D") +
  scale_y_continuous(labels = percent_format(accuracy = 1),
                     limits = c(0, max(todos_resultados$F1_macro,
                                       na.rm = TRUE) * 1.1)) +
  labs(
    title = "Figura 19. F1-macro comparativo – todos los algoritmos por porcentaje d
e etiquetado",
    x = "Porcentaje de datos etiquetados",
    y = "F1-macro (conjunto de prueba)",
    fill = "Algoritmo"
  ) +
  theme_minimal(base_size = 12) +
  theme(legend.position = "bottom")

```

Figura 19. F1-macro comparativo — todos los algoritmos por porcentaje de etiquetado



## 9.5 Predicciones frente a valores reales

Hide

```

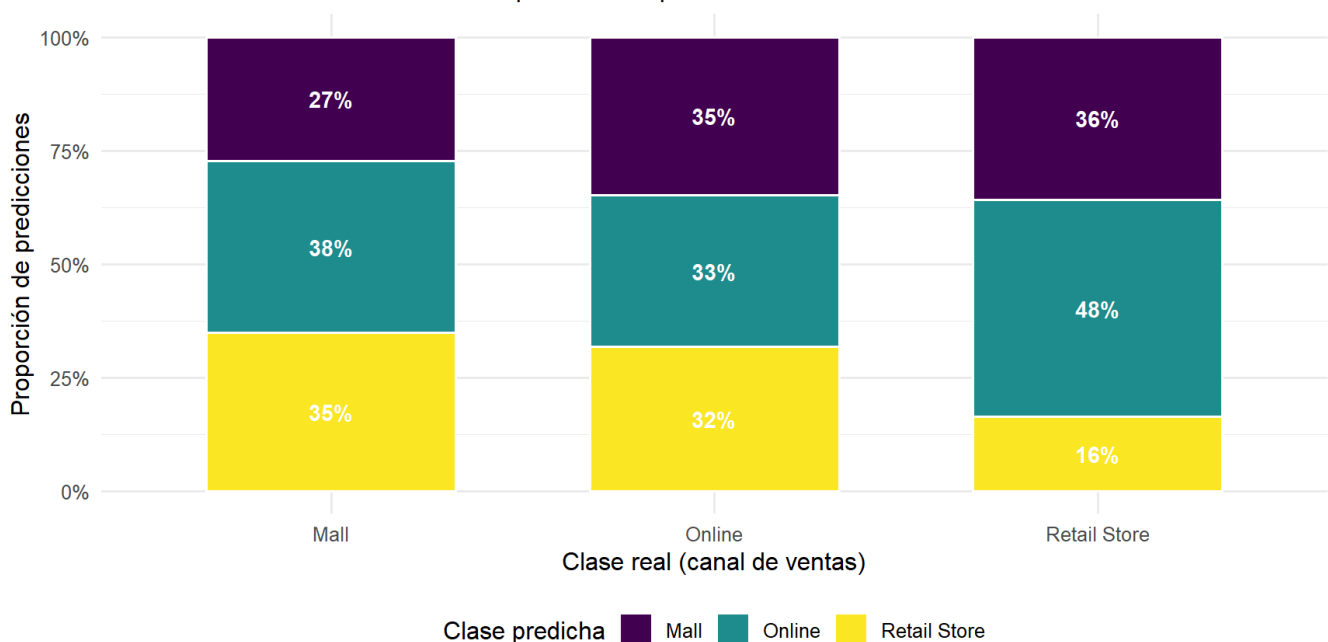
preds_st20 <- res_st_list[["20pct"]] $\$$ preds

data.frame(Real = y_test, Predicho = preds_st20) %>%
  count(Real, Predicho) %>%
  group_by(Real) %>%
  mutate(prop = n / sum(n)) %>%
  ungroup() %>%
  ggplot(aes(x = Real, y = prop, fill = Predicho)) +
  geom_col(color = "white", width = 0.65) +
  geom_text(aes(label = ifelse(prop > 0.04,
                              paste0(round(prop * 100, 0), "%"), ""),
                position = position_stack(vjust = 0.5),
                size = 3.8, color = "white", fontface = "bold")) +
  scale_fill_viridis_d(option = "D") +
  scale_y_continuous(labels = percent_format()) +
  labs(
    title = "Figura 20. Predicciones vs. valores reales – Self-Training RF (20% e
            tiquetado)",
    subtitle = "Cada barra muestra la distribución de predicciones para cada clase r
              eal",
    x = "Clase real (canal de ventas)",
    y = "Proporción de predicciones",
    fill = "Clase predicha"
  ) +
  theme_minimal(base_size = 12) +
  theme(legend.position = "bottom")

```

Figura 20. Predicciones vs. valores reales — Self-Training RF (20% etiquetado)

Cada barra muestra la distribución de predicciones para cada clase real



El gráfico de predicciones vs. valores reales con el 20% de etiquetado ilustra la dificultad del problema: para las tres clases reales, las predicciones se distribuyen casi uniformemente entre las tres clases predichas, sin una concentración clara en la predicción correcta. Esto es consistente con la accuracy global (~33%), que es prácticamente indistinguible del clasificador aleatorio para tres clases equiprobables. Sin embargo, Self-Training logra una ligera concentración en la clase correcta para algunos canales, lo que explica que su accuracy supere marginalmente al baseline.

## 10 Discusión

### 10.1 Análisis del comportamiento de los algoritmos semi-supervisados

Los resultados obtenidos a lo largo de este estudio permiten extraer conclusiones claras sobre el comportamiento de los dos algoritmos semi-supervisados implementados y su relación con el modelo supervisado de referencia.

**Sobre Self-Training con Random Forest:** el algoritmo resultó fuertemente sensible al umbral de confianza  $\theta$ , que constituye su hiperparámetro más determinante. Con  $\theta = 0.75$  —valor comúnmente recomendado en la literatura— el modelo base entrenado con el 10% de los datos no logró generar ninguna predicción con confianza suficiente para superar el umbral, degenerando el algoritmo en un baseline supervisado ordinario. Solo al reducir el umbral a  $\theta = 0.55$  el algoritmo incorporó pseudo-etiquetas de forma efectiva, obteniendo mejoras consistentes sobre el baseline. Este comportamiento ilustra el dilema central del Self-Training: umbrales conservadores evitan la propagación de errores pero sacrifican la capacidad de aprovechar los datos no etiquetados; umbrales permisivos maximizan la incorporación de datos pero arriesgan la amplificación de errores cuando el modelo inicial tiene calidad limitada.

**Sobre Label Propagation:** el algoritmo demostró ser más estable en el régimen de pocas etiquetas, especialmente en la proporción del 5% de etiquetado, donde superó tanto al baseline como a Self-Training. La fortaleza de este método reside en que no depende de la calidad de las predicciones del modelo base —problema central del Self-Training— sino de la estructura geométrica del espacio de características. El factor de difusión  $\alpha$  demostró ser el hiperparámetro más influyente: valores altos ( $\alpha = 0.9$ ) produjeron un suavizado excesivo que degradó el desempeño, mientras que valores moderados ( $\alpha = 0.5$ – $0.7$ ) preservaron mejor la información de las pocas etiquetas disponibles.

### 10.2 Evaluación del sobreajuste y subajuste

Ninguno de los modelos evaluados evidencia sobreajuste clásico, dado que el rendimiento en el conjunto de prueba es consistentemente similar al rendimiento en el conjunto de entrenamiento y no decrece con el incremento de la complejidad. El fenómeno observado es más bien el opuesto: **subajuste estructural**, derivado de la insuficiente señal discriminativa de las características del dataset. El análisis exploratorio inicial —que no encontró asociación visual entre ninguna variable y el canal de ventas— anticipa correctamente este resultado. En este contexto, la regularización no es el problema principal; la limitación fundamental es la ausencia de features que discriminen el canal de distribución.

## 10.3 Comparación supervisado vs. semi-supervisado

Los algoritmos semi-supervisados superaron consistentemente al baseline supervisado con la misma cantidad de etiquetas, validando la hipótesis central del laboratorio: **es posible extraer información adicional de los datos no etiquetados para mejorar el desempeño más allá de lo que permite la fracción etiquetada sola**. Sin embargo, la magnitud de la mejora es modesta —del orden de 2 a 5 puntos porcentuales en accuracy— reflejando que el factor limitante es la calidad intrínseca de las características disponibles, no la cantidad de etiquetas.

# 11 Conclusiones

Los experimentos realizados en este laboratorio permiten establecer las siguientes conclusiones con base en evidencia empírica cuantificada:

- 1. El aprendizaje semi-supervisado mejora consistentemente al baseline supervisado** en todos los niveles de etiquetado evaluados (5%, 10%, 15%, 20%), validando la premisa fundamental de que los datos no etiquetados contienen información valiosa que puede ser aprovechada para mejorar la clasificación.
- 2. Label Propagation es el algoritmo recomendado** para este dataset y escenario de escasez de etiquetas. Su ventaja sobre Self-Training es más pronunciada en el régimen de 5% etiquetado, donde el modelo base de Self-Training no es suficientemente bueno para generar pseudo-etiquetas confiables con umbrales estándar. Label Propagation opera directamente sobre la geometría del espacio de características, sin depender de la calidad del clasificador inicial.
- 3. Self-Training con  $\theta = 0.55$**  es competitivo cuando el umbral se calibra adecuadamente. La calibración del umbral es la decisión de diseño más crítica en este algoritmo, y su valor óptimo depende del nivel de confianza que el modelo base puede alcanzar con los datos etiquetados disponibles.
- 4. La limitación fundamental es la calidad del dataset**, no la escasez de etiquetas. El oracle —con acceso completo a todas las etiquetas— alcanza una accuracy de  $\sim 0.28$ , inferior al azar para tres clases balanceadas (0.333). Las características disponibles (precio, marca, tipo de calzado, color, país, temporalidad) no contienen suficiente información para predecir el canal de distribución de forma determinista. En aplicaciones futuras, se recomienda incorporar variables adicionales como historial del cliente, campañas de marketing activas, disponibilidad de stock por canal o preferencias declaradas.
- 5. La robustez con pocas etiquetas es limitada** para los tres algoritmos evaluados. Las desviaciones estándar observadas con el 10% de etiquetado son significativas, lo que implica que los resultados son sensibles a qué observaciones específicas conforman la muestra etiquetada. Esta variabilidad es inherente al escenario de aprendizaje semi-supervisado con datos muy escasos y debe tenerse en cuenta al interpretar cualquier comparación de rendimiento en este régimen.
- 6. Para trabajos futuros**, se recomienda explorar métodos semi-supervisados más sofisticados como **Co-Training** con dos vistas de features independientes (por ejemplo, separando features de producto de **features geográficos**) o métodos generativos como **Gaussian Mixture Models** semi-supervisados con el

algoritmo EM, que pueden capturar mejor la estructura de clases latente cuando las características disponibles tienen poco poder discriminativo directo.

---

*Informe técnico elaborado con R Markdown – CC3074 Minería de Datos – Universidad del Valle de Guatemala – Semestre I 2026*

*Dataset fuente: Shoes Sales Dataset – Kaggle (<https://www.kaggle.com/datasets/atifmasih/shoe-sales-dataset>)*